

BỘ GIÁO DỤC VÀ ĐÀO TẠO

TRƯỜNG ĐẠI HỌC MỞ THÀNH PHỐ HỒ CHÍ MINH

BÁO CÁO TỔNG KẾT

ĐỀ TÀI BÀI TẬP LỚN MÔN TRÍ TUỆ NHÂN TẠO

Credit Card Fraud Detection Classification

ITEC3413

TP HCM, 5/2026

BỘ GIÁO DỤC VÀ ĐÀO TẠO

TRƯỜNG ĐẠI HỌC MỞ THÀNH PHỐ HỒ CHÍ MINH

BÁO CÁO TỔNG KẾT

ĐỀ TÀI BÀI TẬP LỚN MÔN TRÍ TUỆ NHÂN TẠO

Credit Card Fraud Detection Classification

ITEC3413

Trưởng nhóm: Bùi Nhật Huy

Khoa: Công Nghệ Thông Tin

Thành viên: Lê Hoàng Phúc

Thành viên: Phạm Hoàng Duy

Thành viên: Trần Đăng Khoa

TP HCM, 5/2026

THÔNG TIN KẾT QUẢ NGHIÊN CỨU CỦA ĐỀ TÀI

1. Thông tin chung

- Tên đề tài: Credit Card Fraud Detection Classification
- Lớp: ITEC3413
- Nhóm: 14
- Đề tài 8: Credit Card Fraud Detection (Classification - Imbalanced)
- Nhóm trưởng: Bùi Nhật Huy
- Mã số sinh viên: 2351050051

Tên thành viên	Mã số sinh viên
Lê Hoàng Phúc	2351050136
Phạm Hoàng Duy	2351050021
Trần Đăng Khoa	2351050083

2. Mục tiêu đề tài

Mục tiêu của đề tài là xây dựng và đánh giá các mô hình học máy nhằm phát hiện gian lận trong giao dịch thẻ tín dụng. Bên cạnh đó, đề tài tập trung xử lý dữ liệu mất cân bằng, tối ưu hóa hiệu năng mô hình thông qua các chỉ số phù hợp như AUC-PR và chi phí sai lầm, đồng thời cải thiện độ tin cậy của xác suất dự đoán thông qua các kỹ thuật hiệu chỉnh.

3. Kết quả nghiên cứu

Đề tài đã xây dựng và đánh giá các mô hình học máy cho bài toán phát hiện gian lận, trong đó Random Forest và ensemble LightGBM + Random Forest đạt hiệu năng tốt nhất. Kết quả cho thấy việc xử lý dữ liệu mất cân bằng và tối ưu ngưỡng phân loại giúp cải thiện đáng kể khả năng phát hiện gian lận, đồng thời đảm bảo chi phí sai lầm ở mức hợp lý.

MỤC LỤC

Chương 1. Giới thiệu	8
1.1. Đặt vấn đề	8
1.2. Mục tiêu đề tài	8
1.3. Phạm vi	9
Chương 2. Tổng quan & Cơ sở lý thuyết	10
2.1. Bài toán phát hiện gian lận	10
2.2. Đặc điểm dữ liệu (mất cân bằng)	11
2.3. Các mô hình sử dụng	13
2.3.1. Logistic Regression	13
2.3.2. Random Forest	13
2.3.3. XGBoost	14
2.3.4. LightGBM	15
2.4. Các chỉ số đánh giá (Precision, Recall, AUC, ...)	16
Chương 3. Phân tích dữ liệu & Tiền xử lý	18
3.1. Giới thiệu dataset	18
3.2. Phân tích dữ liệu (EDA)	20
3.3. Xử lý dữ liệu	27
3.3.1. Cleaning	27
3.3.2. Feature Engineering	28
3.4. Xử lý mất cân bằng (Sampling)	30
Chương 4. Xây dựng mô hình	31
4.1. Huấn luyện các mô hình	31
4.1.1. Logistic Regression	31
4.1.2. Random Forest	33
4.1.3. XGBoost	33

4.1.4. LightGBM	35
4.2. Đánh giá mô hình	36
4.3. Tối ưu tham số	36
4.3.1. Tối ưu tham số Logistic Regression	36
4.3.2. Tối ưu tham số Random Forest	38
4.3.3. Tối ưu tham số XGBoost	38
4.3.4. Tối ưu tham số LightGBM	39
4.4. Ensemble (kết hợp mô hình)	40
4.4.1. LightGBM + Random Forest	40
4.4.2. XGBoost + Random Forest	41
4.4.3. LightGBM + XGBoost	42
4.5. Phân tích Precision-Recall Curve	43
Chương 5. Triển khai hệ thống	48
5.1. Tổng quan hệ thống	48
5.2. Backend (API)	48
5.3. Frontend	49
5.4. Demo hệ thống	50

Danh Sách Hình

Hình 1: Biểu đồ phân tán giữa V17 và V14	19
Hình 2: Biểu đồ phân tán giữa V17 và V12	19
Hình 3: Phân bố lớp (Class Distribution – Log Scale)	20
Hình 4: Tỷ lệ gian lận theo phân vị số tiền giao dịch (10 nhóm)	21
Hình 5: Phân bố giá trị giao dịch (Amount) theo lớp	22
Hình 6: Phân bố log(Amount) theo lớp	22
Hình 7: Phân bố các đặc trưng PCA (V1–V28) theo lớp	23
Hình 8: Headmap của 6 biến quan trọng	24
Hình 9: Features with Correlation > 0.1	24
Hình 10: Fraud Rate by Four	25
Hình 11: Transactions per Hour	26
Hình 12: Fraud Rate by Time of Day	26
Hình 13: Fraud Detection - PR Curves (Test)	44
Hình 14: Fraud - Reliability Overlay (Test) bins=10, strategy=quantile	45
Hình 15: XGB - Cost & Alerts vs Threshold (Test)	46
Hình 16: RF - Cost & Alerts vs Threshold (Test)	46
Hình 17: LGBM - Cost & Alerts vs Threshold (Test)	46
Hình 18: Ensemble LGBM + RF - Cost & Alerts vs Threshold (Test)	47
Hình 19: Ensemble RF + XGB - Cost & Alerts vs Threshold (Test)	47
Hình 20: Ensemble XGB + LGBM - Cost & Alerts vs Threshold (Test)	47
Hình 21: Ảnh Demo 1	50
Hình 22: Ảnh Demo 2	50
Hình 23: Ảnh Demo 3	51
Hình 24: Ảnh Demo 4	51
Hình 25: Ảnh Demo 5	52

Hình 26: Ảnh Demo 6	52
Hình 27: Ảnh Demo 7	53
Hình 28: Ảnh Demo 8	53

Chương 1. Giới thiệu

1.1. Đặt vấn đề

Trong bối cảnh chuyển đổi số diễn ra mạnh mẽ, các giao dịch tài chính điện tử, đặc biệt là giao dịch thẻ tín dụng, ngày càng trở nên phổ biến và đóng vai trò quan trọng trong hệ sinh thái thanh toán hiện đại. Tuy nhiên, song song với sự phát triển này, các hành vi gian lận (fraud) cũng ngày càng tinh vi, gây ra những tổn thất đáng kể cho cả ngân hàng lẫn người sử dụng. Theo thực tế, các giao dịch gian lận thường chiếm tỷ lệ rất nhỏ trong tổng số giao dịch, chỉ khoảng 0,1% đến 0,2%, nhưng lại mang mức độ rủi ro và thiệt hại tài chính rất lớn.

Chính đặc điểm phân bố dữ liệu mất cân bằng nghiêm trọng (imbalanced data) đã khiến bài toán phát hiện gian lận thẻ tín dụng trở nên đặc biệt thách thức. Các mô hình học máy truyền thống thường có xu hướng thiên lệch về lớp chiếm đa số (giao dịch hợp lệ), từ đó dẫn đến việc bỏ sót các giao dịch gian lận – điều không thể chấp nhận trong các hệ thống tài chính thực tế. Do đó, yêu cầu đặt ra không chỉ dừng lại ở việc nâng cao độ chính xác tổng thể (accuracy), mà quan trọng hơn là tối ưu khả năng phát hiện gian lận (Recall), đồng thời vẫn kiểm soát tỷ lệ cảnh báo sai (False Positive) ở mức hợp lý.

Bên cạnh đó, dữ liệu giao dịch thẻ tín dụng thường bao gồm nhiều đặc trưng đã được xử lý hoặc biến đổi (chẳng hạn như các đặc trưng ẩn danh hóa thông qua PCA), cùng với các thuộc tính cơ bản như thời gian giao dịch (Time) và giá trị giao dịch (Amount). Điều này đòi hỏi quá trình xây dựng mô hình không chỉ dừng lại ở việc lựa chọn thuật toán, mà còn cần chú trọng đến bước tiền xử lý dữ liệu, xây dựng đặc trưng (feature engineering) và xử lý mất cân bằng thông qua các kỹ thuật như oversampling (SMOTE), undersampling hoặc điều chỉnh trọng số lớp (class_weight).

Xuất phát từ những vấn đề nêu trên, đề tài “Credit Card Fraud Detection” được thực hiện nhằm nghiên cứu và xây dựng các mô hình phân loại có khả năng phát hiện hiệu quả các giao dịch gian lận trong điều kiện dữ liệu mất cân bằng cao. Đồng thời, đề tài cũng hướng đến việc đánh giá mô hình thông qua các thước đo phù hợp với bài toán thực tế như AUC-PR, Recall và các tiêu chí liên quan đến chi phí sai lầm (cost-sensitive evaluation), từ đó góp phần hỗ trợ ra quyết định trong các hệ thống phát hiện gian lận trong thực tiễn.

1.2. Mục tiêu đề tài

Trên cơ sở những thách thức đã được trình bày, đề tài “Credit Card Fraud Detection” hướng đến việc xây dựng một hệ thống phát hiện gian lận giao dịch thẻ tín dụng dựa trên các phương pháp học máy, nhằm giải quyết hiệu quả bài toán phân loại trong điều kiện dữ liệu mất cân bằng nghiêm trọng. Cụ thể, mục tiêu trọng tâm của đề tài là phát triển các mô hình có khả năng phân biệt chính xác giữa giao dịch hợp lệ và giao dịch gian lận dựa trên các đặc trưng của dữ liệu giao dịch, bao gồm các biến đã được xử lý như các thành phần PCA cùng với các thuộc tính cơ bản như thời gian và giá trị giao dịch.

Trước hết, đề tài tập trung vào việc nghiên cứu và áp dụng các kỹ thuật xử lý dữ liệu mất cân bằng, bao gồm các phương pháp lấy mẫu lại (sampling) như SMOTE, undersampling, cũng như cơ chế điều chỉnh trọng số lớp trong các mô hình học máy. Mục tiêu của bước này là cải thiện khả năng học của mô hình đối với lớp thiểu số (giao dịch gian lận), từ đó hạn chế tình trạng bỏ sót các trường hợp gian lận – yếu tố mang ý nghĩa quan trọng trong bối cảnh ứng dụng thực tế.

Tiếp theo, đề tài hướng đến việc xây dựng và so sánh nhiều mô hình phân loại khác nhau, từ các phương pháp cơ bản đến các mô hình nâng cao, nhằm lựa chọn được mô hình tối ưu cho bài toán. Quá trình đánh giá mô hình không chỉ dựa trên các chỉ số truyền thống như Accuracy hay ROC-AUC, mà đặc biệt chú trọng đến các thước đo phù hợp với dữ liệu mất cân bằng như AUC-PR (Area Under Precision-Recall Curve) và Recall, đồng thời kết hợp với các tiêu chí đánh giá dựa trên chi phí (cost-sensitive metrics) để phản ánh đúng tác động thực tế của các loại sai sót.

Ngoài ra, đề tài cũng đặt mục tiêu tối ưu hóa ngưỡng phân loại (threshold tuning) nhằm cân bằng giữa khả năng phát hiện gian lận và tỷ lệ cảnh báo sai, từ đó nâng cao hiệu quả vận hành của hệ thống trong môi trường thực tế. Cuối cùng, toàn bộ quá trình từ tiền xử lý dữ liệu, xây dựng mô hình đến đánh giá và lựa chọn mô hình sẽ được tổ chức theo một quy trình chặt chẽ, đảm bảo tính tái lập (reproducibility) và khả năng triển khai trong các hệ thống phát hiện gian lận thực tế.

1.3. Phạm vi

Trong khuôn khổ của đề tài “Credit Card Fraud Detection”, phạm vi nghiên cứu được xác định nhằm tập trung giải quyết bài toán phát hiện gian lận trong giao dịch thẻ tín dụng dưới góc độ học máy, đồng thời đảm bảo tính khả thi trong việc triển khai và đánh giá mô hình.

Trước hết, về phạm vi dữ liệu, đề tài sử dụng tập dữ liệu giao dịch thẻ tín dụng được công bố trên nền tảng Kaggle, bao gồm khoảng 284.000 giao dịch với các đặc trưng đã được xử lý và ẩn danh hóa thông qua phương pháp phân tích thành phần chính (PCA), cùng với các thuộc tính cơ bản như thời gian giao dịch (Time) và giá trị giao dịch (Amount). Dữ liệu này có đặc điểm nổi bật là mất cân bằng nghiêm trọng giữa hai lớp, trong đó các giao dịch gian lận chỉ chiếm tỷ lệ rất nhỏ, tạo điều kiện phù hợp để nghiên cứu các kỹ thuật xử lý imbalanced data.

Về phạm vi phương pháp, đề tài tập trung vào các mô hình học máy có giám sát (supervised learning) cho bài toán phân loại nhị phân, bao gồm các thuật toán tiêu biểu như Logistic Regression, Random Forest, Gradient Boosting (điển hình là XGBoost, LightGBM), cùng với các phương pháp hiệu chỉnh xác suất (calibration) và kết hợp mô hình (ensemble). Bên cạnh đó, các kỹ thuật xử lý mất cân bằng như oversampling (SMOTE), undersampling và điều chỉnh trọng số lớp cũng được xem xét và đánh giá nhằm cải thiện hiệu năng mô hình.

Ngoài ra, đề tài giới hạn trong việc đánh giá mô hình dựa trên các thước đo phù hợp với bài toán mất cân bằng, đặc biệt là AUC-PR, Recall, Precision, cùng với các chỉ số liên quan đến độ hiệu chỉnh xác suất như Brier Score và Expected Calibration Error (ECE). Đồng thời, một số tiêu chí đánh giá dựa trên chi phí sai lầm (cost-sensitive evaluation) cũng được áp dụng để phản ánh sát hơn các yêu cầu trong thực tế vận hành hệ thống phát hiện gian lận.

Tuy nhiên, đề tài không đi sâu vào việc triển khai hệ thống phát hiện gian lận theo thời gian thực (real-time system) trên hạ tầng sản xuất, cũng như không xử lý các vấn đề liên quan đến bảo mật dữ liệu, tích hợp hệ thống ngân hàng hay các yếu tố pháp lý. Mục tiêu chính vẫn tập trung vào việc xây dựng, phân tích và đánh giá mô hình ở mức nghiên cứu và thực nghiệm, làm nền tảng cho các hướng phát triển và triển khai trong tương lai.

Chương 2. Tổng quan & Cơ sở lý thuyết

2.1. Bài toán phát hiện gian lận

Bài toán phát hiện gian lận trong giao dịch thẻ tín dụng là một dạng điển hình của bài toán phân loại nhị phân trong lĩnh vực học máy, trong đó mỗi giao dịch được gán vào một trong hai lớp: giao dịch hợp lệ (legitimate) hoặc giao dịch gian lận (fraud). Mục tiêu cốt lõi của bài toán là xây dựng một mô hình có khả năng nhận diện chính

xác các giao dịch gian lận dựa trên các đặc trưng của dữ liệu đầu vào, từ đó hỗ trợ hệ thống tài chính đưa ra các cảnh báo kịp thời.

Tuy nhiên, khác với các bài toán phân loại thông thường, phát hiện gian lận mang những đặc trưng riêng biệt làm gia tăng đáng kể độ khó của bài toán. Trước hết, dữ liệu có tính mất cân bằng nghiêm trọng, khi số lượng giao dịch gian lận chỉ chiếm một tỷ lệ rất nhỏ so với tổng số giao dịch, thường dao động trong khoảng 0,1% đến 0,2% . Điều này dẫn đến việc các mô hình học máy dễ bị thiên lệch về phía lớp đa số, khiến cho việc dự đoán chính xác các trường hợp gian lận trở nên khó khăn.

Bên cạnh đó, hành vi gian lận thường xuyên thay đổi và có tính thích nghi cao, khiến cho các đặc trưng phân biệt giữa giao dịch hợp lệ và gian lận không cố định theo thời gian. Điều này đặt ra yêu cầu đối với mô hình phải có khả năng tổng quát hóa tốt và thích ứng với các mẫu dữ liệu mới. Ngoài ra, dữ liệu giao dịch thường chứa nhiều đặc trưng đã được biến đổi hoặc ẩn danh hóa (ví dụ như các thành phần PCA), làm giảm khả năng diễn giải trực quan nhưng lại giúp bảo vệ thông tin nhạy cảm của người dùng.

Một khía cạnh quan trọng khác của bài toán là sự đánh đổi giữa hai loại sai lầm: bỏ sót giao dịch gian lận (False Negative) và cảnh báo nhầm giao dịch hợp lệ (False Positive). Trong thực tế, việc bỏ sót một giao dịch gian lận thường gây ra hậu quả nghiêm trọng hơn nhiều so với việc cảnh báo nhầm, do đó các hệ thống phát hiện gian lận thường ưu tiên tối đa hóa Recall cho lớp gian lận, đồng thời vẫn cần kiểm soát tỷ lệ False Positive ở mức chấp nhận được để tránh ảnh hưởng đến trải nghiệm người dùng.

Ngoài ra, bài toán phát hiện gian lận còn gắn liền với yêu cầu ra quyết định theo ngưỡng (threshold-based decision). Thay vì chỉ dựa vào xác suất dự đoán, hệ thống cần lựa chọn một ngưỡng phân loại phù hợp để chuyển đổi xác suất thành quyết định nhị phân. Việc lựa chọn ngưỡng này thường phụ thuộc vào mục tiêu kinh doanh và chi phí của từng loại sai lầm, do đó việc tối ưu hóa ngưỡng đóng vai trò quan trọng trong việc nâng cao hiệu quả tổng thể của hệ thống.

Từ những đặc điểm trên có thể thấy rằng, bài toán phát hiện gian lận không chỉ đơn thuần là một bài toán phân loại, mà còn là một bài toán tối ưu đa mục tiêu, đòi hỏi sự kết hợp giữa kỹ thuật xử lý dữ liệu, lựa chọn mô hình, đánh giá hiệu năng và cân nhắc yếu tố chi phí trong thực tế.

2.2. Đặc điểm dữ liệu (mất cân bằng)

Một trong những đặc điểm quan trọng và mang tính quyết định đối với bài toán phát hiện gian lận thẻ tín dụng là hiện tượng mất cân bằng dữ liệu (class imbalance). Cụ thể, trong tập dữ liệu giao dịch, số lượng giao dịch hợp lệ (legitimate) chiếm tỷ lệ áp đảo, trong khi các giao dịch gian lận (fraud) chỉ chiếm một phần rất nhỏ, thường dao động trong khoảng 0,1% đến 0,2% tổng số giao dịch. Sự chênh lệch lớn này tạo ra nhiều thách thức trong việc xây dựng và đánh giá mô hình học máy.

Trước hết, trong bối cảnh dữ liệu mất cân bằng, các thước đo truyền thống như Accuracy không còn phản ánh đúng hiệu năng của mô hình. Một mô hình đơn giản chỉ dự đoán tất cả các giao dịch là hợp lệ vẫn có thể đạt độ chính xác rất cao, nhưng lại hoàn toàn thất bại trong việc phát hiện gian lận. Do đó, việc đánh giá mô hình cần chuyển sang các thước đo phù hợp hơn như Precision, Recall, đặc biệt là AUC-PR, nhằm tập trung vào khả năng nhận diện lớp thiểu số.

Bên cạnh đó, dữ liệu mất cân bằng còn ảnh hưởng trực tiếp đến quá trình học của mô hình. Các thuật toán học máy có xu hướng tối ưu hàm mất mát tổng thể, dẫn đến việc ưu tiên dự đoán đúng lớp đa số và bỏ qua các mẫu thuộc lớp thiểu số. Điều này khiến mô hình dễ rơi vào tình trạng “bias” đối với lớp hợp lệ, làm giảm đáng kể khả năng phát hiện các giao dịch gian lận.

Ngoài ra, trong bối cảnh thực tế, chi phí của các loại sai lầm là không đồng đều. Việc bỏ sót một giao dịch gian lận (False Negative) có thể gây ra thiệt hại tài chính trực tiếp, trong khi việc cảnh báo nhầm một giao dịch hợp lệ (False Positive) chủ yếu ảnh hưởng đến trải nghiệm người dùng. Do đó, dữ liệu mất cân bằng không chỉ là vấn đề về phân phối dữ liệu, mà còn liên quan chặt chẽ đến bài toán tối ưu chi phí (cost-sensitive learning), trong đó mô hình cần được thiết kế để ưu tiên phát hiện lớp gian lận.

Để giải quyết vấn đề này, nhiều kỹ thuật đã được đề xuất và áp dụng. Các phương pháp lấy mẫu lại dữ liệu như oversampling (ví dụ SMOTE) giúp tăng số lượng mẫu của lớp thiểu số, trong khi undersampling làm giảm số lượng mẫu của lớp đa số nhằm cân bằng lại phân phối dữ liệu. Bên cạnh đó, các phương pháp điều chỉnh trọng số lớp (class_weight) cũng được sử dụng để tăng mức độ “quan tâm” của mô hình đối với các mẫu gian lận trong quá trình huấn luyện.

Tóm lại, đặc điểm mất cân bằng dữ liệu không chỉ là một thuộc tính của tập dữ liệu, mà còn là yếu tố cốt lõi chi phối toàn bộ quy trình xây dựng mô hình, từ bước tiền

xử lý, lựa chọn thuật toán để đánh giá và tối ưu hóa. Việc hiểu rõ và xử lý hiệu quả đặc điểm này đóng vai trò then chốt trong việc nâng cao hiệu quả của hệ thống phát hiện gian lận trong thực tế.

2.3. Các mô hình sử dụng

2.3.1. Logistic Regression

Logistic Regression là một trong những mô hình học máy cơ bản và phổ biến nhất được sử dụng cho bài toán phân loại nhị phân. Trong bối cảnh phát hiện gian lận thẻ tín dụng, mô hình này được sử dụng để ước lượng xác suất một giao dịch thuộc lớp gian lận (fraud) hoặc hợp lệ (legitimate) dựa trên các đặc trưng đầu vào.

Khác với Linear Regression, Logistic Regression sử dụng hàm sigmoid để ánh xạ giá trị đầu ra về khoảng $[0,1]$, từ đó có thể diễn giải như một xác suất

$$P(y = 1|x) = \frac{1}{1+e^{-(w^T x+b)}} ,$$

trong đó, w là vector trọng số, x là vector đặc trưng đầu vào và b là hệ số bias. Mô hình sẽ dự đoán một giao dịch là gian lận nếu xác suất đầu ra vượt qua một ngưỡng nhất định (threshold), thường mặc định là 0.5, tuy nhiên có thể điều chỉnh tùy theo mục tiêu bài toán.

Ưu điểm của Logistic Regression là đơn giản, dễ triển khai, tốc độ huấn luyện nhanh và có khả năng diễn giải cao thông qua các hệ số của mô hình. Điều này giúp người dùng hiểu được mức độ ảnh hưởng của từng đặc trưng đến khả năng xảy ra gian lận.

Tuy nhiên, mô hình này có một số hạn chế, đặc biệt là giả định mối quan hệ tuyến tính giữa các đặc trưng và log-odds của biến mục tiêu. Trong các bài toán phức tạp như phát hiện gian lận, nơi tồn tại nhiều mối quan hệ phi tuyến, Logistic Regression có thể không đạt hiệu năng cao như các mô hình nâng cao như Random Forest hay boosting.

Ngoài ra, trong bối cảnh dữ liệu mất cân bằng, Logistic Regression cần được kết hợp với các kỹ thuật như điều chỉnh trọng số lớp (class_weight) hoặc thay đổi ngưỡng phân loại nhằm cải thiện khả năng phát hiện các giao dịch gian lận.

2.3.2. Random Forest

Random Forest là một thuật toán học máy thuộc nhóm ensemble, trong đó nhiều cây quyết định (decision trees) được xây dựng và kết hợp trong quá trình huấn

luyện nhằm cải thiện hiệu năng dự đoán. Mỗi cây được huấn luyện trên một tập con ngẫu nhiên của dữ liệu (bootstrap sampling), đồng thời tại mỗi nút phân tách, mô hình chỉ xem xét một tập con ngẫu nhiên các đặc trưng. Cơ chế này giúp tăng tính đa dạng giữa các cây và giảm hiện tượng overfitting.

Random Forest có thể được áp dụng cho cả bài toán phân loại (classification) và hồi quy (regression). Đối với bài toán phân loại, kết quả cuối cùng được xác định dựa trên nguyên tắc bỏ phiếu đa số (majority voting) từ các cây quyết định. Trong khi đó, đối với bài toán hồi quy, kết quả được tính bằng trung bình giá trị dự đoán của tất cả các cây trong mô hình.

Nhờ cơ chế kết hợp nhiều mô hình con, Random Forest giúp giảm phương sai (variance) và hạn chế ảnh hưởng của các dự đoán bất thường (outliers), từ đó cải thiện độ ổn định và khả năng tổng quát hóa của mô hình. Ngoài ra, thuật toán này còn cho phép ước lượng mức độ quan trọng của các đặc trưng (feature importance), hỗ trợ việc phân tích và giải thích kết quả dự đoán.

2.3.3. XGBoost

XGBoost (eXtreme Gradient Boosting) là một thuật toán học máy thuộc nhóm ensemble learning, cụ thể là phương pháp boosting dựa trên cây quyết định. Khác với các mô hình đơn lẻ, XGBoost xây dựng một tập hợp các cây quyết định theo cơ chế học tuần tự, trong đó mỗi cây mới được thêm vào nhằm khắc phục sai số của các cây trước đó, từ đó cải thiện dần hiệu năng tổng thể của mô hình.

Về mặt lý thuyết, XGBoost dựa trên nguyên lý của Gradient Boosting – một kỹ thuật học tăng cường trong đó mô hình được xây dựng bằng cách tối ưu hóa hàm mất mát thông qua phương pháp gradient descent trong không gian hàm. Điểm nổi bật của XGBoost là việc sử dụng khai triển Taylor bậc hai của hàm mất mát để xấp xỉ quá trình tối ưu. Thay vì chỉ sử dụng gradient bậc nhất, thuật toán còn khai thác thêm thông tin từ đạo hàm bậc hai (Hessian), giúp quá trình học hội tụ nhanh hơn và đạt độ chính xác cao hơn.

Trong quá trình huấn luyện, tại mỗi vòng lặp, mô hình bổ sung một cây quyết định mới sao cho làm giảm hàm mục tiêu nhiều nhất. Việc xây dựng cây được thực hiện thông qua việc tìm kiếm điểm chia tối ưu dựa trên giá trị gain, từ đó xác định cấu trúc phù hợp cho từng cây.

Bên cạnh đó, XGBoost tích hợp nhiều cơ chế tối ưu quan trọng như xử lý giá trị thiếu tự động, cắt tỉa cây (tree pruning), lấy mẫu theo cột (column subsampling) và hỗ trợ tính toán song song. Những đặc điểm này giúp mô hình không chỉ đạt hiệu năng cao mà còn đảm bảo tốc độ huấn luyện nhanh và khả năng tổng quát hóa tốt.

2.3.4. LightGBM

LightGBM (Light Gradient Boosting Machine) là một thuật toán học máy thuộc nhóm phương pháp boosting dựa trên cây quyết định, được phát triển nhằm tối ưu hóa hiệu năng và tốc độ huấn luyện trên các tập dữ liệu có kích thước lớn. Thuật toán này kế thừa ý tưởng từ Gradient Boosting truyền thống, trong đó các mô hình yếu (weak learners), thường là các cây quyết định (decision trees), được xây dựng tuần tự nhằm giảm thiểu sai số của mô hình tổng thể thông qua việc tối ưu hàm mất mát.

Điểm nổi bật của LightGBM nằm ở cách tiếp cận xây dựng cây theo chiều sâu (leaf-wise growth) thay vì theo từng mức (level-wise) như các thuật toán boosting truyền thống. Cụ thể, tại mỗi bước, LightGBM sẽ lựa chọn nút lá có mức giảm hàm mất mát lớn nhất để tiếp tục phân tách, từ đó giúp mô hình hội tụ nhanh hơn và đạt được độ chính xác cao hơn. Tuy nhiên, cơ chế này cũng có thể làm tăng nguy cơ overfitting nếu không được kiểm soát thông qua các tham số như độ sâu tối đa (max_depth) hoặc số lượng mẫu tối thiểu tại mỗi nút (min_child_samples).

Bên cạnh đó, LightGBM sử dụng kỹ thuật histogram-based để rời rạc hóa các giá trị liên tục thành các bins, từ đó giảm đáng kể chi phí tính toán và bộ nhớ trong quá trình huấn luyện. Ngoài ra, hai kỹ thuật quan trọng khác được tích hợp trong LightGBM là Gradient-based One-Side Sampling (GOSS) và Exclusive Feature Bundling (EFB). Trong đó, GOSS giúp giữ lại các mẫu có gradient lớn (mang nhiều thông tin) và giảm số lượng mẫu có gradient nhỏ, còn EFB cho phép gộp các đặc trưng thưa (sparse features) lại với nhau nhằm giảm số chiều của dữ liệu.

Trong bài toán phát hiện gian lận thẻ tín dụng, LightGBM tỏ ra đặc biệt hiệu quả nhờ khả năng xử lý dữ liệu lớn, tốc độ huấn luyện nhanh và khả năng nắm bắt các mối quan hệ phi tuyến giữa các đặc trưng. Hơn nữa, thuật toán này hỗ trợ cơ chế điều chỉnh trọng số lớp (class_weight), giúp cải thiện hiệu năng trên dữ liệu mất cân bằng – một đặc điểm điển hình của bài toán fraud detection. Đồng thời, LightGBM cũng cho phép tối ưu trực tiếp theo các hàm mục tiêu phù hợp như AUC hoặc các biến thể liên quan đến precision-recall, từ đó nâng cao khả năng phát hiện lớp gian lận.

Tuy nhiên, việc áp dụng LightGBM cũng đòi hỏi quá trình tinh chỉnh siêu tham số (hyperparameter tuning) một cách cẩn thận, bao gồm các tham số như số lượng cây (`n_estimators`), tốc độ học (`learning_rate`), số lá (`num_leaves`) và các tham số điều chỉnh regularization. Nếu được cấu hình hợp lý, LightGBM có thể trở thành một trong những mô hình mạnh mẽ và hiệu quả nhất cho bài toán phát hiện gian lận trong điều kiện dữ liệu mất cân bằng cao.

2.4. Các chỉ số đánh giá (Precision, Recall, AUC, ...)

Trong bài toán phát hiện gian lận thể tín dụng, việc lựa chọn các chỉ số đánh giá phù hợp không chỉ dừng lại ở việc mô tả ý nghĩa, mà còn cần hiểu rõ cách tính toán thông qua các công thức toán học. Điều này giúp đảm bảo quá trình đánh giá mô hình mang tính chính xác và có cơ sở lý thuyết vững chắc, đặc biệt trong bối cảnh dữ liệu mất cân bằng.

Trước hết, các chỉ số cơ bản được xây dựng dựa trên ma trận nhầm lẫn (confusion matrix), bao gồm bốn thành phần: True Positive (TP), True Negative (TN), False Positive (FP) và False Negative (FN). Trong đó, TP là số giao dịch gian lận được dự đoán đúng, FP là số giao dịch hợp lệ nhưng bị dự đoán nhầm là gian lận, FN là số giao dịch gian lận bị bỏ sót, và TN là số giao dịch hợp lệ được dự đoán đúng.

Từ các thành phần này, Precision được xác định theo công thức

$$\text{Precision} = \frac{TP}{(TP + FP)} ,$$

chỉ số này phản ánh tỷ lệ các dự đoán gian lận là chính xác, do đó càng cao thì hệ thống càng đáng tin cậy trong việc đưa ra cảnh báo.

Trong khi đó, Recall được định nghĩa là

$$\text{Recall} = \frac{TP}{(TP + FN)} ,$$

Recall thể hiện khả năng phát hiện các giao dịch gian lận thực sự, do đó trong bài toán fraud detection, đây là chỉ số đặc biệt quan trọng cần được tối ưu.

Để cân bằng giữa Precision và Recall, F1-score được sử dụng với công thức trung bình điều hòa

$$F1 = \frac{2 \times (\text{Precision} \times \text{Recall})}{(\text{Precision} + \text{Recall})} ,$$

F1-score cho phép đánh giá tổng thể hiệu năng mô hình khi hai chỉ số Precision và Recall có sự đánh đổi với nhau.

Bên cạnh đó, độ chính xác tổng thể (Accuracy) được tính như sau

$$\text{Accuracy} = \frac{(TP + TN)}{(TP + TN + FP + FN)} ,$$

tuy nhiên, như đã đề cập, Accuracy không phải là chỉ số phù hợp trong trường hợp dữ liệu mất cân bằng, vì có thể gây ra nhận định sai lệch về hiệu năng mô hình.

Đối với các chỉ số dựa trên đường cong, ROC-AUC được xây dựng từ mối quan hệ giữa True Positive Rate (TPR) và False Positive Rate (FPR), trong đó

$$\text{TPR} = \frac{TP}{(TP + FN)} ,$$

$$\text{FPR} = \frac{FP}{(FP + TN)} ,$$

ROC-AUC đo diện tích dưới đường cong ROC, phản ánh khả năng phân biệt giữa hai lớp trên toàn bộ các ngưỡng phân loại. Tuy nhiên, trong bài toán mất cân bằng, AUC-PR (Area Under Precision-Recall Curve) thường được ưu tiên hơn do tập trung trực tiếp vào hiệu năng trên lớp gian lận.

Ngoài các chỉ số phân loại, việc đánh giá xác suất dự đoán cũng rất quan trọng. Brier Score được sử dụng để đo lường sai số giữa xác suất dự đoán và giá trị thực tế

$$\text{Brier Score} = \frac{1}{N} \sum_{i=1}^N (p - y)^2 ,$$

trong đó p là xác suất dự đoán và y là nhãn thực tế. Giá trị Brier Score càng nhỏ thì mô hình càng tốt.

Ngoài ra, Expected Calibration Error (ECE) được sử dụng để đánh giá mức độ hiệu chỉnh của mô hình

$$\text{ECE} = \sum_m \frac{|B|}{N} \cdot |acc(B) - conf(B)| ,$$

trong đó B là tập mẫu thuộc bin thứ m , $acc(B)$ là độ chính xác trong bin, $conf(B)$ là xác suất trung bình của bin, và N là tổng số mẫu.

ECE càng nhỏ thì mô hình càng được hiệu chỉnh tốt. Tuy nhiên, ECE truyền thống phụ thuộc vào cách chia bin nên có thể không ổn định khi dữ liệu mất cân bằng. Để khắc phục, Adaptive ECE sử dụng phương pháp chia bin theo tần suất (equal-frequency binning), giúp mỗi bin có số lượng mẫu tương đối đồng đều, từ đó giảm phương sai trong đánh giá. Bên cạnh đó, Debiased ECE được đề xuất nhằm giảm sai lệch thống kê khi số lượng mẫu trong từng bin nhỏ, giúp ước lượng chính xác hơn mức độ calibration.

Trong bài toán phát hiện gian lận thẻ tín dụng, các chỉ số calibration như ECE, Adaptive ECE và Debiased ECE đặc biệt quan trọng, vì hệ thống không chỉ cần dự

đoán đúng mà còn cần cung cấp xác suất đáng tin cậy để phục vụ cho việc thiết lập ngưỡng quyết định (threshold tuning) và tối ưu chi phí. Một mô hình có AUC-PR cao nhưng calibration kém có thể dẫn đến quyết định sai lệch khi triển khai thực tế.

Tóm lại, việc kết hợp giữa các chỉ số phân loại, chỉ số xác suất và đánh giá theo chi phí, cùng với việc hiểu rõ công thức toán học của chúng, sẽ giúp xây dựng một hệ thống đánh giá toàn diện và phù hợp cho bài toán phát hiện gian lận thẻ tín dụng.

Chương 3. Phân tích dữ liệu & Tiền xử lý

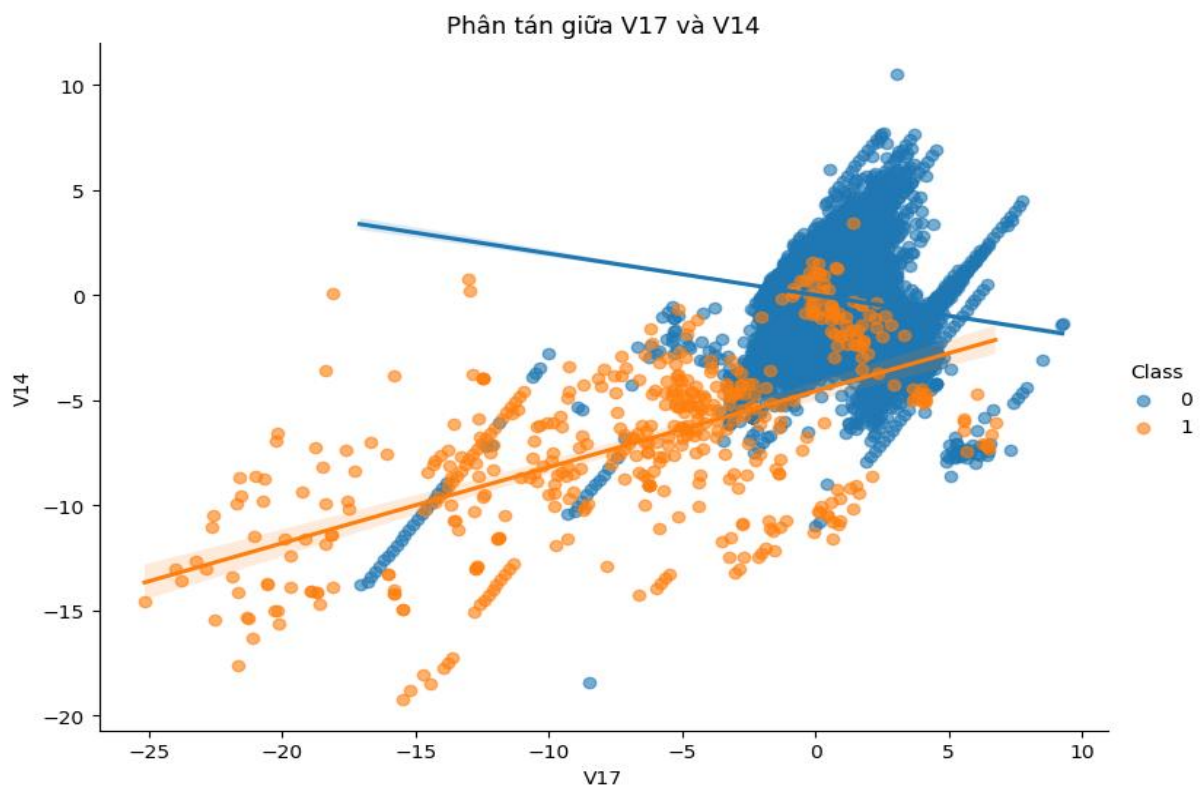
3.1. Giới thiệu dataset

Trong khuôn khổ đề tài “Credit Card Fraud Detection”, tập dữ liệu được sử dụng là bộ dữ liệu giao dịch thẻ tín dụng công khai trên nền tảng Kaggle, bao gồm các giao dịch thực tế của chủ thẻ tại châu Âu. Tập dữ liệu này được sử dụng rộng rãi trong các nghiên cứu về phát hiện gian lận do có quy mô lớn, tính thực tiễn cao và đặc biệt phù hợp với bài toán phân loại trong điều kiện dữ liệu mất cân bằng.

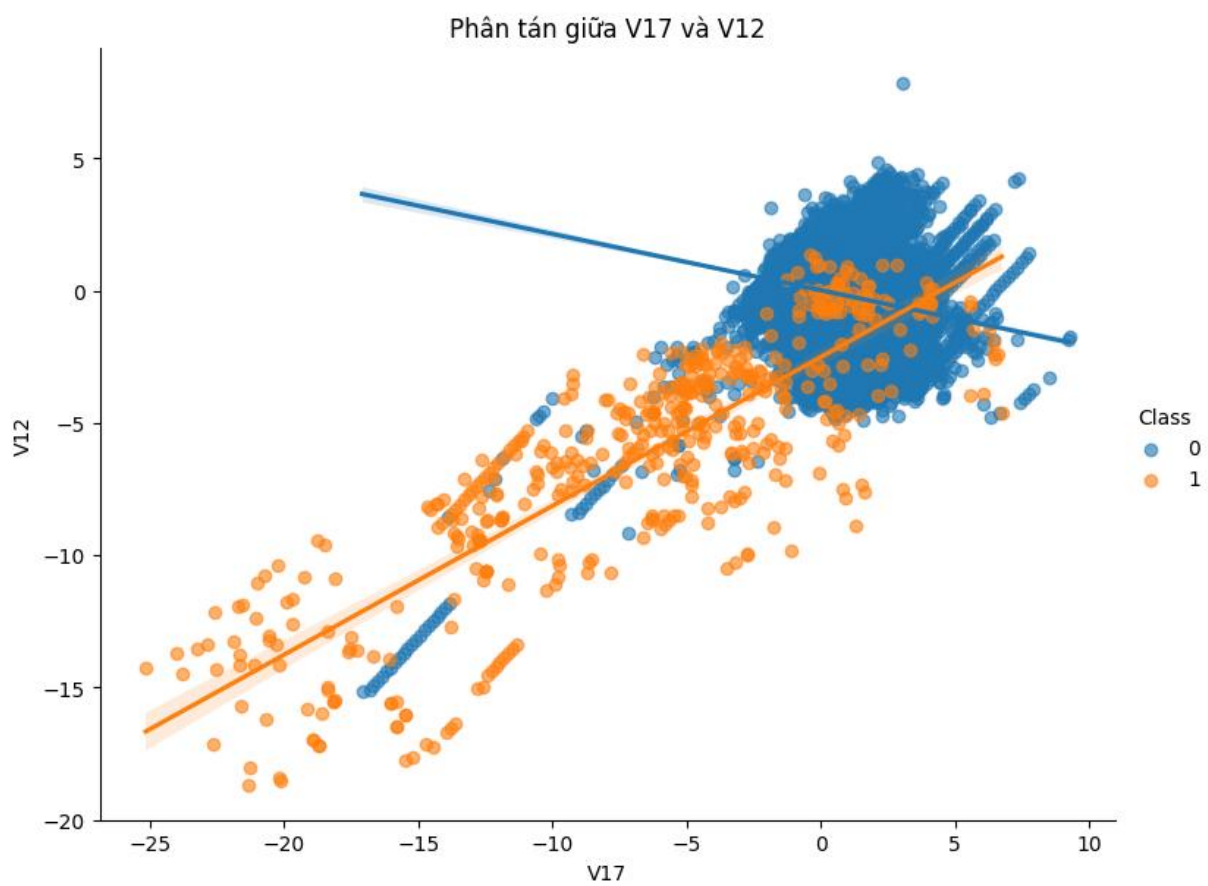
Cụ thể, tập dữ liệu bao gồm khoảng 284.000 giao dịch, trong đó mỗi dòng dữ liệu tương ứng với một giao dịch riêng biệt. Mỗi giao dịch được mô tả thông qua một tập hợp các đặc trưng đầu vào, bao gồm hai thuộc tính cơ bản là thời gian giao dịch (Time) và giá trị giao dịch (Amount), cùng với 28 đặc trưng ẩn danh (V1 đến V28). Các đặc trưng này đã được biến đổi thông qua kỹ thuật phân tích thành phần chính (Principal Component Analysis – PCA) nhằm bảo vệ thông tin nhạy cảm của khách hàng, đồng thời giữ lại các thông tin quan trọng phục vụ cho việc phân tích và xây dựng mô hình.

Biến mục tiêu (target variable) trong bài toán là biến “Class”, trong đó giá trị 0 đại diện cho giao dịch hợp lệ và giá trị 1 đại diện cho giao dịch gian lận. Như đã đề cập, dữ liệu có sự mất cân bằng nghiêm trọng giữa hai lớp, khi số lượng giao dịch gian lận chỉ chiếm một tỷ lệ rất nhỏ so với tổng số giao dịch. Đây là một đặc điểm quan trọng cần được xem xét xuyên suốt quá trình phân tích dữ liệu và xây dựng mô hình.

Ngoài ra, một điểm đáng chú ý của tập dữ liệu là các đặc trưng V1–V28 không mang ý nghĩa trực quan do đã được biến đổi, điều này làm hạn chế khả năng diễn giải trực tiếp nhưng lại giúp mô hình học được các biểu diễn trừu tượng có giá trị. Do đó, quá trình phân tích dữ liệu trong đề tài chủ yếu tập trung vào việc khảo sát phân phối dữ liệu, kiểm tra sự mất cân bằng, cũng như đánh giá mối quan hệ giữa các đặc trưng và biến mục tiêu thông qua các phương pháp thống kê và trực quan hóa.



Hình 1: Biểu đồ phân tán giữa V17 và V14



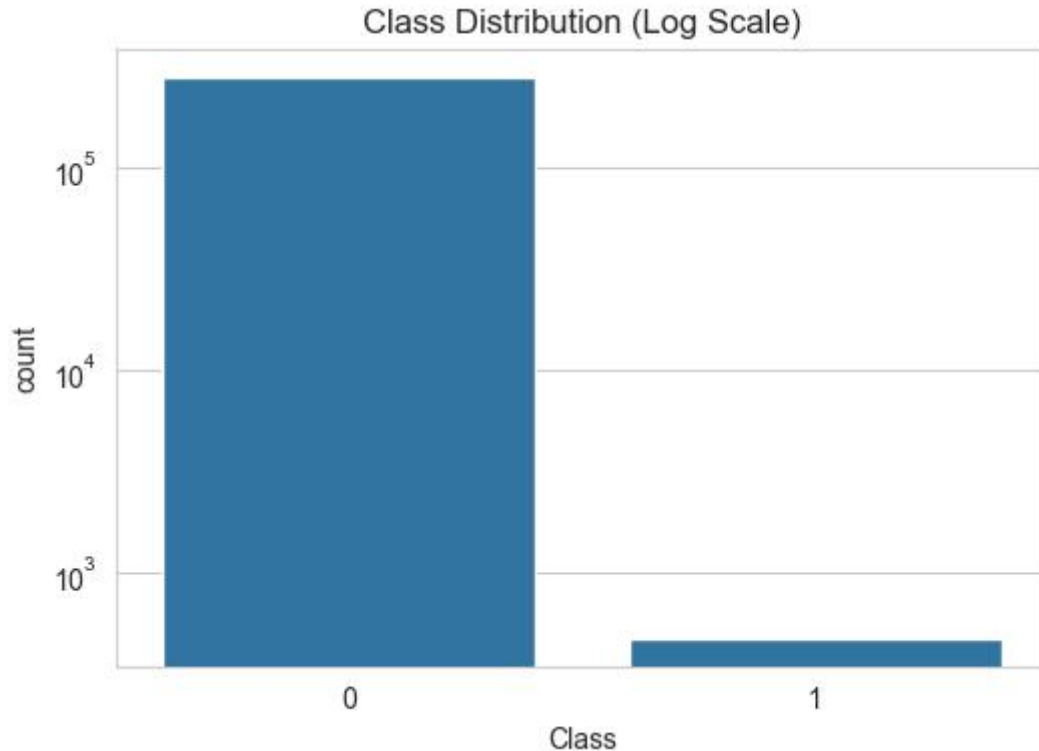
Hình 2: Biểu đồ phân tán giữa V17 và V12

Tóm lại, tập dữ liệu được lựa chọn không chỉ đáp ứng yêu cầu về quy mô và tính thực tiễn, mà còn cung cấp một môi trường phù hợp để nghiên cứu các kỹ thuật xử lý dữ liệu mất cân bằng và xây dựng mô hình phát hiện gian lận hiệu quả. Đây là nền tảng quan trọng cho các bước tiền xử lý và phát triển mô hình trong các phần tiếp theo của đề tài.

3.2. Phân tích dữ liệu (EDA)

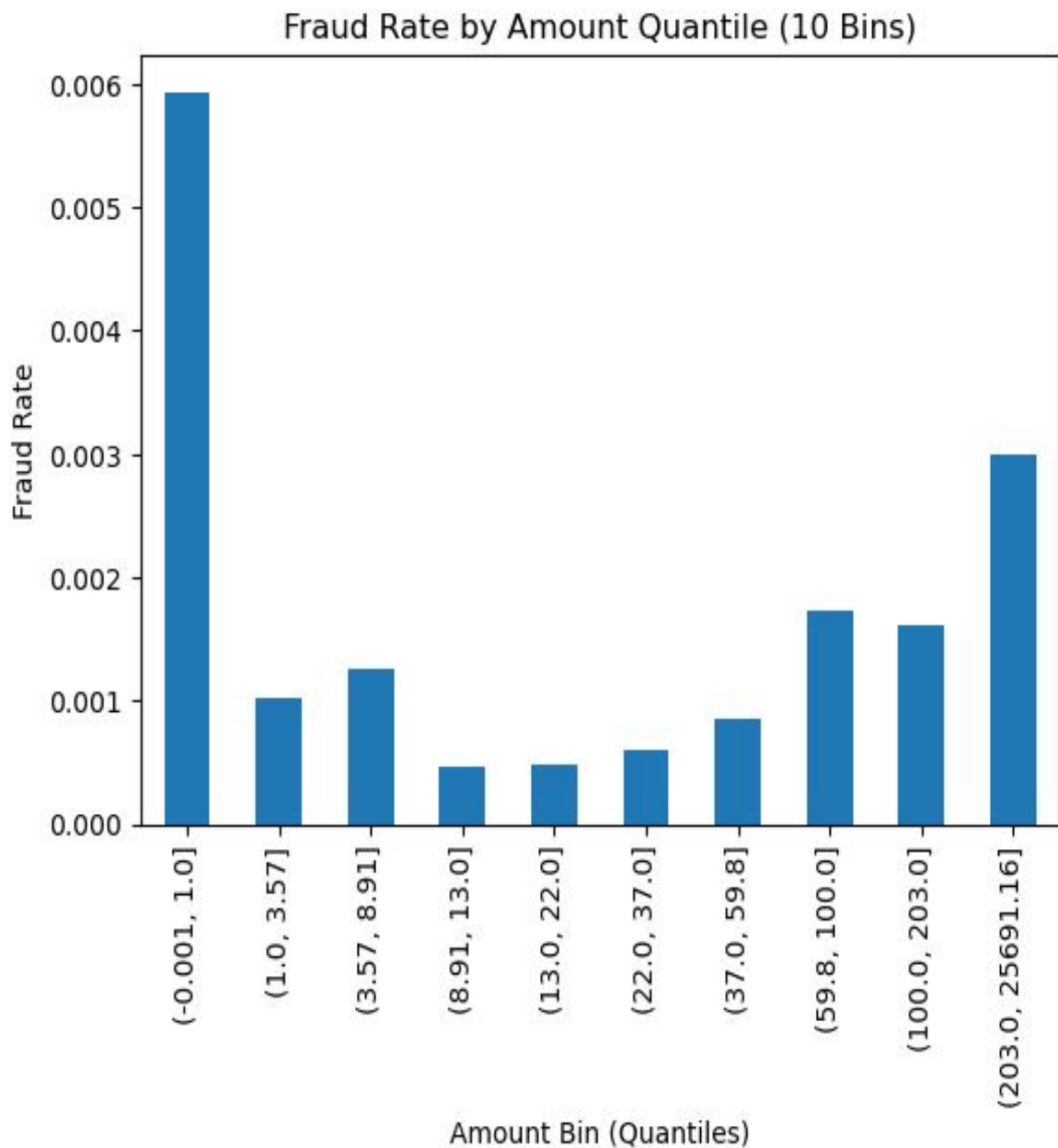
Sau khi giới thiệu tổng quan về tập dữ liệu, bước tiếp theo trong quy trình là thực hiện phân tích khám phá dữ liệu (Exploratory Data Analysis – EDA) nhằm hiểu rõ đặc điểm phân bố, cấu trúc và mối quan hệ giữa các biến. Đây là bước quan trọng giúp định hướng cho quá trình tiền xử lý và xây dựng mô hình ở các giai đoạn tiếp theo.

Trước hết, việc phân tích phân bố của biến mục tiêu (Class) cho thấy sự mất cân bằng nghiêm trọng giữa hai lớp, trong đó số lượng giao dịch gian lận chiếm tỷ lệ rất nhỏ so với tổng số giao dịch. Điều này xác nhận lại bản chất của bài toán và nhấn mạnh sự cần thiết của việc sử dụng các kỹ thuật xử lý imbalanced data cũng như lựa chọn các chỉ số đánh giá phù hợp như AUC-PR và Recall.

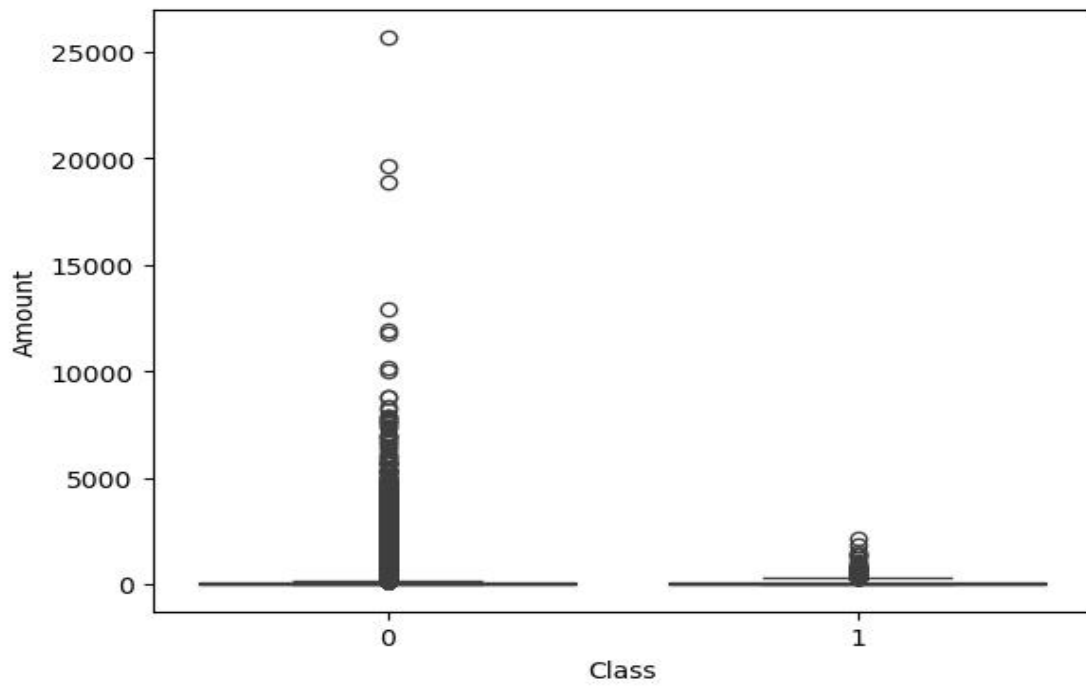


Hình 3: Phân bố lớp (Class Distribution – Log Scale)

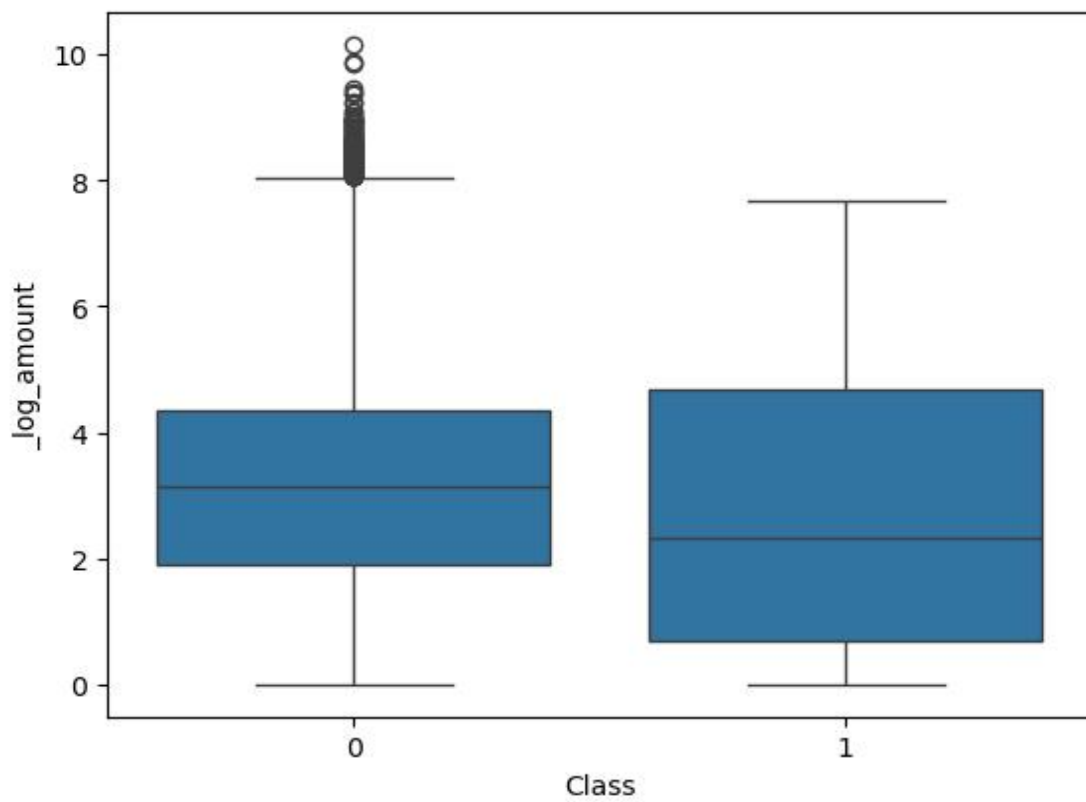
Tiếp theo, đối với các đặc trưng đầu vào, biến Amount được phân tích nhằm hiểu rõ phân bố giá trị giao dịch. Thông thường, dữ liệu này có xu hướng lệch phải (right-skewed), với phần lớn giao dịch có giá trị nhỏ và một số ít giao dịch có giá trị rất lớn. Do đó, việc áp dụng các phép biến đổi như log transformation (ví dụ: $\log(\text{Amount} + 1)$) là cần thiết để giảm độ lệch và giúp mô hình học tốt hơn. Trong khi đó, biến Time thể hiện thứ tự thời gian của các giao dịch, từ đó có thể được sử dụng để trích xuất thêm các đặc trưng mang tính chu kỳ như giờ trong ngày (hour), phục vụ cho việc phân tích hành vi giao dịch theo thời gian.



Hình 4: Tỷ lệ gian lận theo phân vị số tiền giao dịch (10 nhóm)

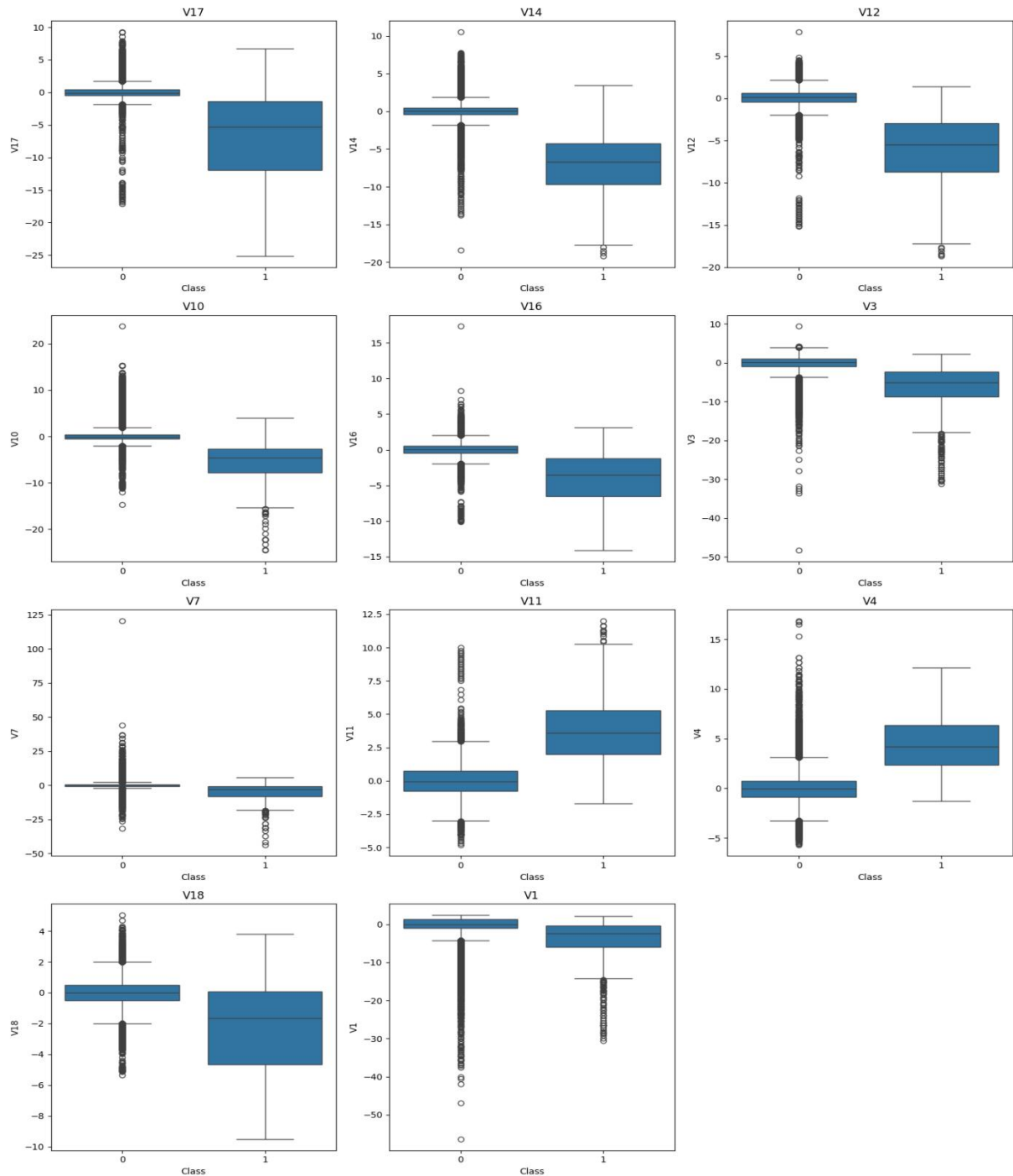


Hình 5: Phân bố giá trị giao dịch (Amount) theo lớp



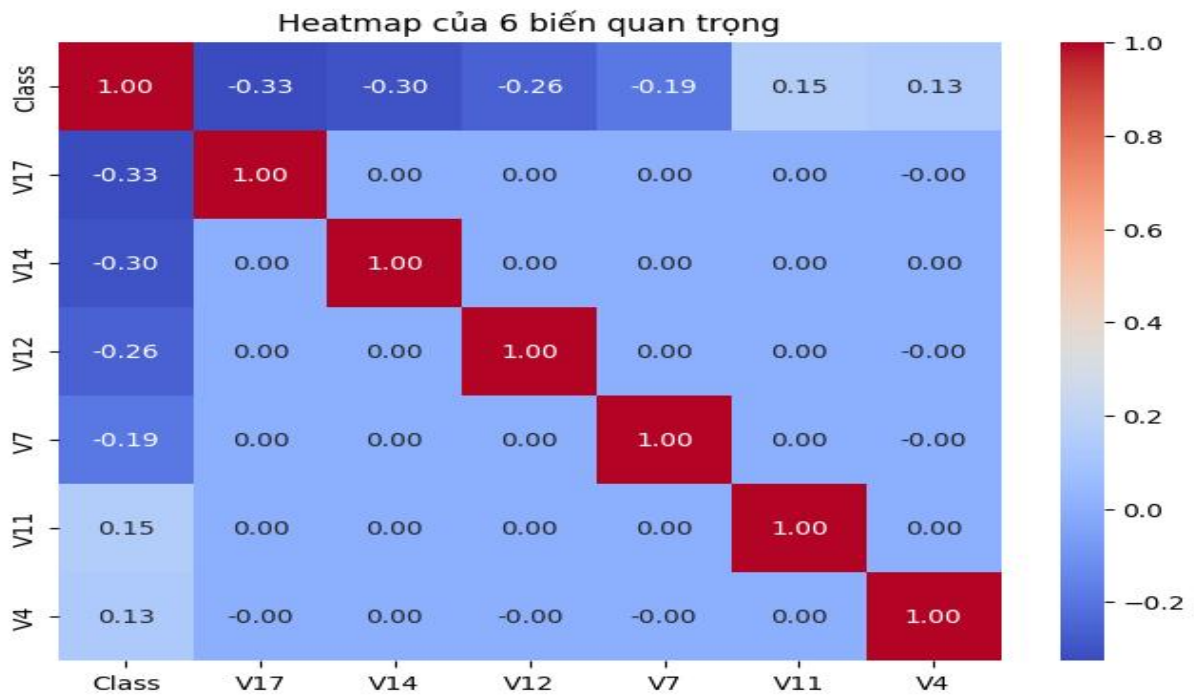
Hình 6: Phân bố $\log(\text{Amount})$ theo lớp

Đối với các đặc trưng V1–V28, do đã được biến đổi thông qua PCA, việc diễn giải trực quan trở nên hạn chế. Tuy nhiên, vẫn có thể tiến hành phân tích thống kê mô tả và trực quan hóa phân phối (histogram, density plot) để quan sát sự khác biệt giữa hai lớp. Ngoài ra, việc sử dụng các kỹ thuật như boxplot hoặc violin plot cho từng đặc trưng theo lớp (fraud vs legitimate) giúp nhận diện các biến có khả năng phân tách tốt giữa hai nhóm.

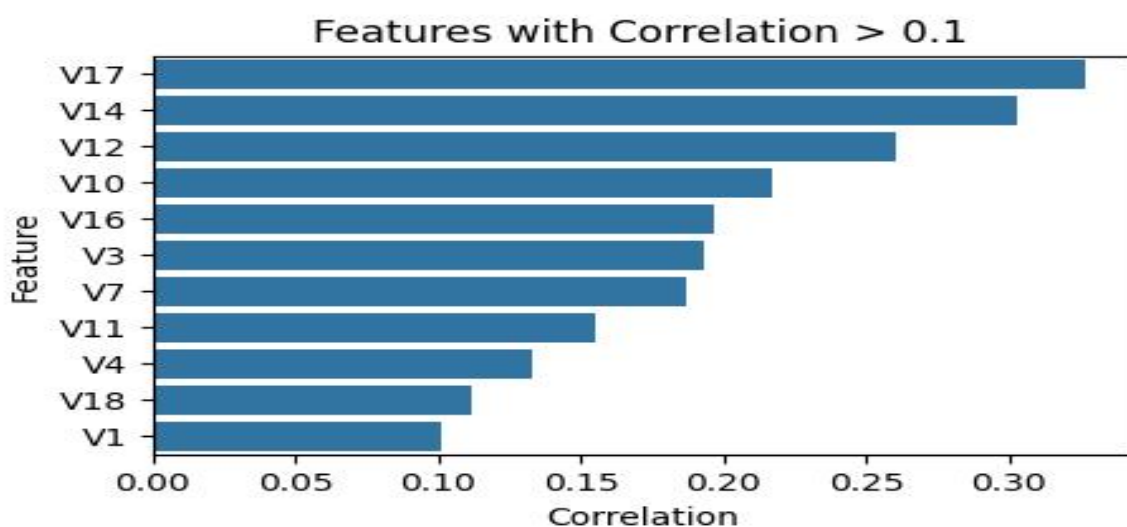


Hình 7: Phân bố các đặc trưng PCA (V1–V28) theo lớp

Bên cạnh đó, phân tích tương quan giữa các đặc trưng cũng được thực hiện nhằm phát hiện các mối quan hệ tuyến tính hoặc phi tuyến tiềm ẩn. Mặc dù các đặc trưng PCA thường có xu hướng ít tương quan với nhau, việc kiểm tra ma trận tương quan vẫn giúp xác nhận giả định này và phát hiện các đặc trưng có thể mang thông tin trùng lặp. Đồng thời, việc đánh giá tương quan giữa từng đặc trưng với biến mục tiêu cũng giúp xác định các yếu tố có ảnh hưởng mạnh đến khả năng xảy ra gian lận.



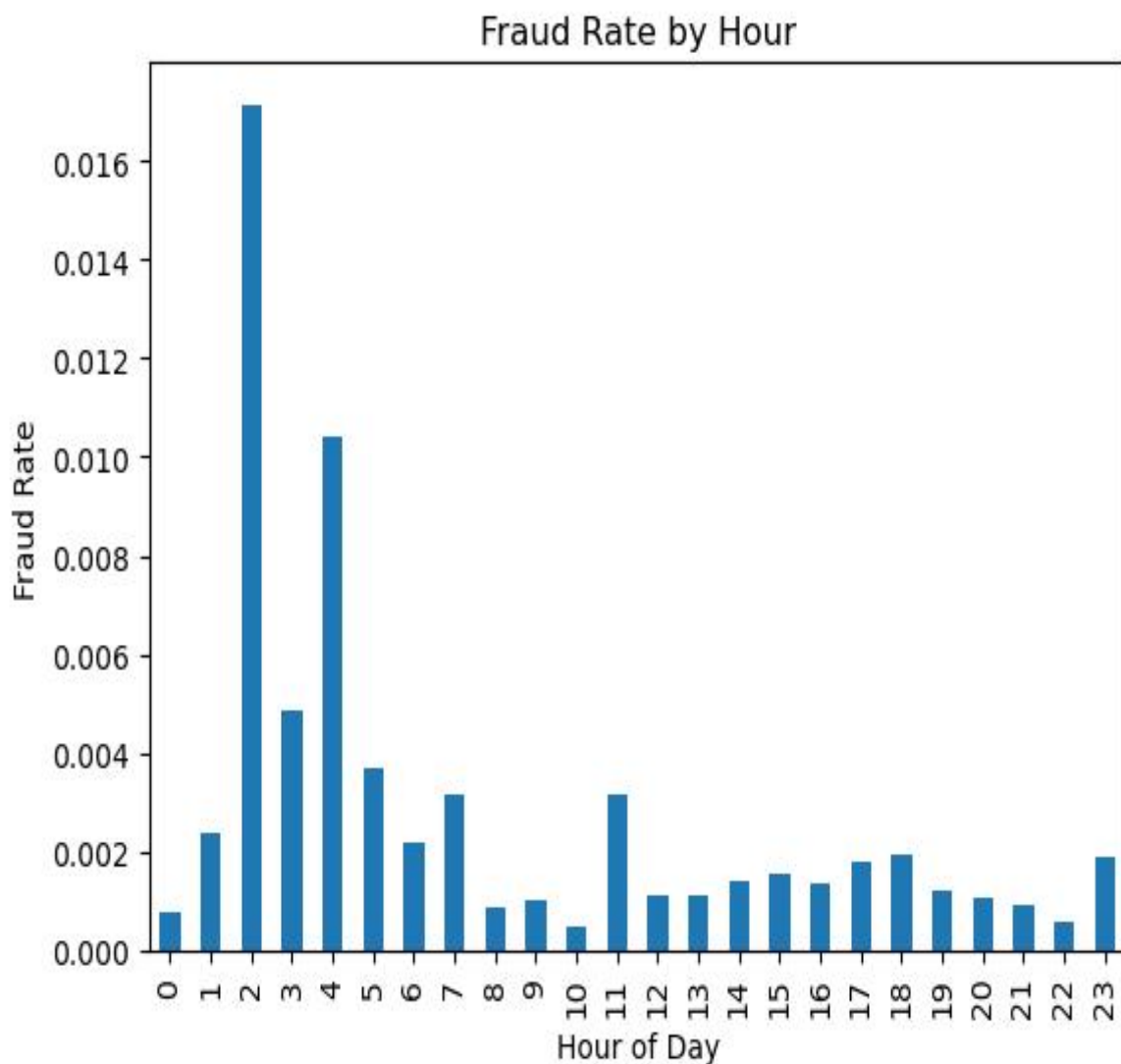
Hình 8: Headmap của 6 biến quan trọng



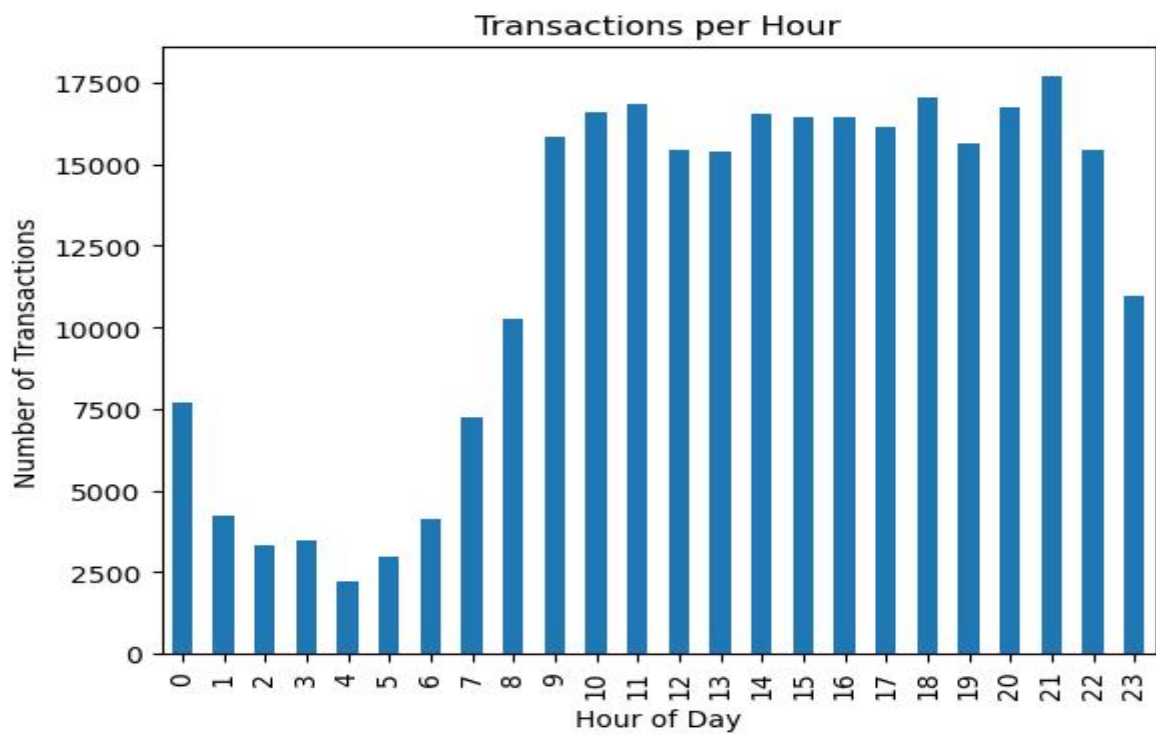
Hình 9: Features with Correlation > 0.1

Ngoài ra, trong quá trình EDA, việc kiểm tra dữ liệu thiếu (missing values) và các giá trị ngoại lai (outliers) cũng được thực hiện. Trong tập dữ liệu này, thông thường không tồn tại giá trị thiếu do đã được xử lý trước, tuy nhiên các giá trị ngoại lai, đặc biệt trong biến Amount, vẫn cần được xem xét và xử lý phù hợp.

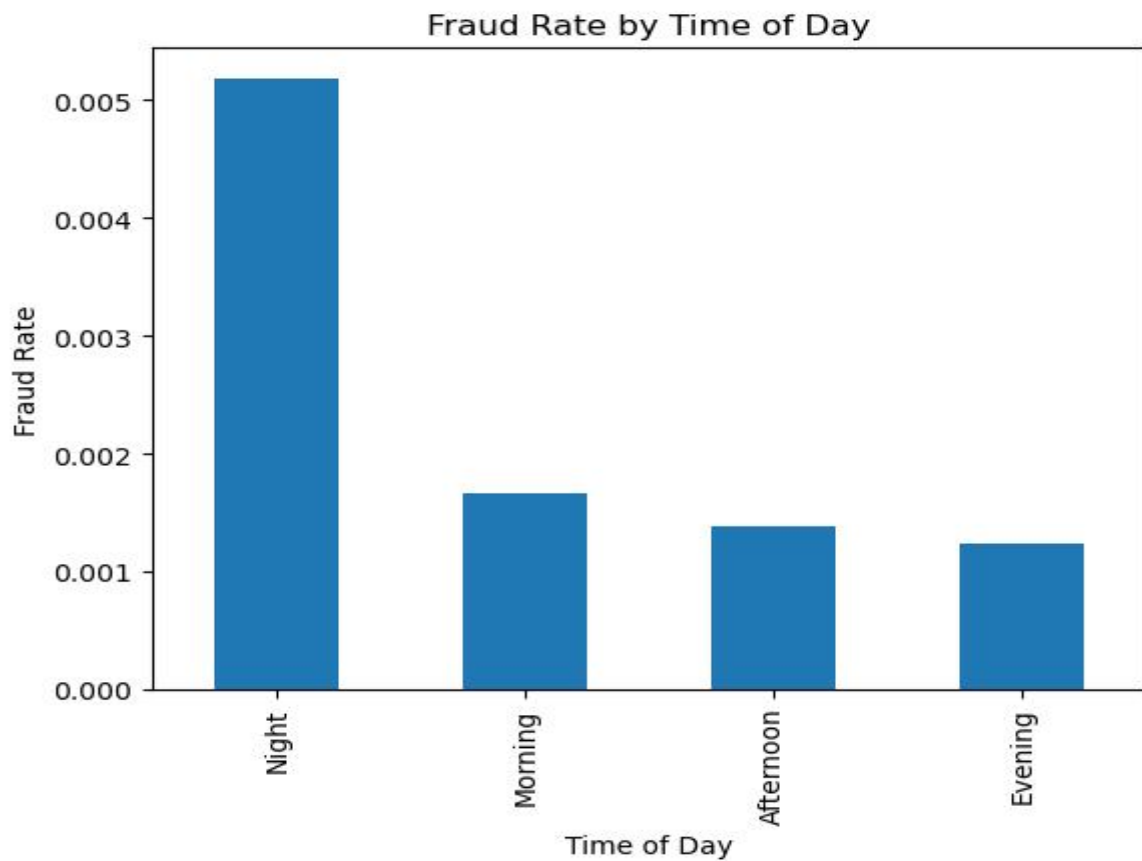
Cuối cùng, một yếu tố quan trọng trong EDA của bài toán này là phân tích theo trục thời gian. Do dữ liệu có tính chất tuần tự, việc sắp xếp theo biến Time và quan sát sự phân bố của các giao dịch gian lận theo thời gian giúp phát hiện các xu hướng hoặc cụm hành vi bất thường. Điều này cũng hỗ trợ cho việc xây dựng chiến lược chia tập dữ liệu theo thời gian (time-based split) nhằm đảm bảo tính thực tế trong quá trình huấn luyện và đánh giá mô hình.



Hình 10: Fraud Rate by Hour



Hình 11: Transactions per Hour



Hình 12: Fraud Rate by Time of Day

Tóm lại, quá trình EDA không chỉ giúp hiểu rõ cấu trúc và đặc điểm của dữ liệu, mà còn cung cấp những insight quan trọng để định hướng cho các bước tiền xử lý và lựa chọn mô hình, góp phần nâng cao hiệu quả của hệ thống phát hiện gian lận.

3.3. Xử lý dữ liệu

3.3.1. Cleaning

Sau bước phân tích khám phá dữ liệu (EDA), quá trình xử lý dữ liệu (data preprocessing) được thực hiện nhằm đảm bảo dữ liệu đầu vào có chất lượng cao, phù hợp cho việc huấn luyện mô hình học máy. Trong đó, bước làm sạch dữ liệu (data cleaning) đóng vai trò nền tảng, giúp loại bỏ các yếu tố gây nhiễu và đảm bảo tính nhất quán của dữ liệu.

Trước hết, việc kiểm tra giá trị thiếu (missing values) được thực hiện trên toàn bộ tập dữ liệu. Đối với bộ dữ liệu sử dụng trong đề tài, các đặc trưng đã được xử lý trước nên hầu như không tồn tại giá trị thiếu. Điều này giúp đơn giản hóa quá trình tiền xử lý và cho phép tập trung vào các bước xử lý nâng cao hơn. Tuy nhiên, việc xác nhận lại tính đầy đủ của dữ liệu vẫn là cần thiết nhằm đảm bảo không có sai sót trong quá trình nhập liệu hoặc xử lý trước đó.

Tiếp theo, vấn đề trùng lặp dữ liệu (duplicate records) cũng được xem xét. Trong bối cảnh giao dịch tài chính, các bản ghi trùng lặp có thể làm sai lệch phân phối dữ liệu và ảnh hưởng đến quá trình học của mô hình. Do đó, việc kiểm tra và loại bỏ các bản ghi trùng lặp (nếu có) là một bước quan trọng nhằm đảm bảo mỗi giao dịch được xem là một quan sát độc lập.

Một khía cạnh quan trọng khác trong quá trình làm sạch dữ liệu là xử lý các giá trị ngoại lai (outliers), đặc biệt đối với biến Amount. Như đã phân tích trong phần EDA, biến này thường có phân phối lệch phải với một số giá trị rất lớn. Thay vì loại bỏ hoàn toàn các giá trị ngoại lai, đề tài áp dụng các phép biến đổi như log transformation (ví dụ: $\log(\text{Amount} + 1)$) nhằm giảm độ lệch của phân phối, giúp mô hình học ổn định hơn mà vẫn giữ được thông tin quan trọng của dữ liệu.

Ngoài ra, việc kiểm tra tính hợp lệ của dữ liệu cũng được thực hiện, bao gồm việc đảm bảo các giá trị của biến Time không bị sai lệch và các đặc trưng V1–V28 nằm trong phạm vi hợp lý sau khi biến đổi PCA. Mặc dù các đặc trưng này không có ý nghĩa trực quan, nhưng việc kiểm tra thống kê cơ bản (mean, standard deviation, min, max) vẫn giúp phát hiện các giá trị bất thường nếu có.

Cuối cùng, một bước quan trọng trong quá trình cleaning là đảm bảo tính nhất quán giữa các biến đầu vào và biến mục tiêu. Điều này bao gồm việc xác nhận rằng biến Class chỉ chứa hai giá trị hợp lệ (0 và 1), không tồn tại các giá trị ngoài phạm vi, cũng như đảm bảo dữ liệu không bị sai lệch trong quá trình gán nhãn.

Tóm lại, mặc dù bộ dữ liệu đã được xử lý ở mức độ nhất định trước khi sử dụng, quá trình làm sạch dữ liệu vẫn đóng vai trò quan trọng trong việc đảm bảo chất lượng đầu vào. Việc kiểm tra và xử lý các vấn đề như giá trị thiếu, trùng lặp, ngoại lai và tính hợp lệ của dữ liệu giúp tạo nền tảng vững chắc cho các bước tiền xử lý và xây dựng mô hình trong các phần tiếp theo.

3.3.2. Feature Engineering

Sau bước làm sạch dữ liệu, quá trình xây dựng đặc trưng (Feature Engineering) được thực hiện nhằm nâng cao khả năng biểu diễn thông tin của dữ liệu đầu vào, từ đó giúp mô hình học máy khai thác tốt hơn các mẫu (patterns) tiềm ẩn trong dữ liệu. Trong bài toán phát hiện gian lận thẻ tín dụng, mặc dù phần lớn các đặc trưng đã được biến đổi thông qua PCA (V1–V28), việc xây dựng thêm các đặc trưng mới vẫn đóng vai trò quan trọng trong việc cải thiện hiệu năng mô hình.

Trước hết, đối với biến Amount, do có phân phối lệch phải, đề tài áp dụng phép biến đổi logarit nhằm giảm độ lệch và ổn định phương sai. Cụ thể, một đặc trưng mới được tạo ra dưới dạng $\log(\text{Amount} + 1)$, thường được ký hiệu là `_log_amount`. Việc sử dụng đặc trưng này giúp mô hình học hiệu quả hơn so với việc sử dụng trực tiếp giá trị gốc của Amount.

Tiếp theo, từ biến Time, đề tài tiến hành trích xuất các đặc trưng liên quan đến yếu tố thời gian. Do Time thể hiện số giây kể từ giao dịch đầu tiên, việc chuyển đổi sang dạng chu kỳ (cyclical features) là cần thiết để phản ánh tính tuần hoàn của thời gian trong ngày. Cụ thể, biến giờ trong ngày được tính từ Time và biểu diễn bằng sin-cos nhằm bảo toàn tính liên tục

$$(\text{hour} = (\text{Time} // 3600) \bmod 24),$$

$$(\text{hour_sin} = \sin(2\pi \cdot \text{hour} / 24)),$$

$$(\text{hour_cos} = \cos(2\pi \cdot \text{hour} / 24)),$$

việc sử dụng biểu diễn sin-cos giúp mô hình nhận diện được tính liên tục giữa các thời điểm như 23h và 0h, điều mà biểu diễn tuyến tính thông thường không thể hiện được.

Ngoài ra, một số đặc trưng heuristic cũng được xây dựng nhằm phản ánh hành vi giao dịch. Ví dụ, các biến hiển nhị phân như `is_night_proxy` hoặc `is_business_hours_proxy` có thể giúp mô hình phát hiện các hành vi bất thường, do các giao dịch gian lận thường có xu hướng xảy ra vào những khung giờ ít phổ biến.

Bên cạnh đó, tùy theo mô hình sử dụng, việc giữ lại hoặc loại bỏ một số đặc trưng gốc cũng được cân nhắc. Ví dụ, khi đã sử dụng `_log_amount`, có thể xem xét loại bỏ `Amount` gốc nếu điều này giúp cải thiện hiệu năng mô hình. Tuy nhiên, quyết định này cần được kiểm chứng thông qua thực nghiệm.

Tổng thể, quá trình feature engineering trong đề tài không chỉ dừng lại ở việc tạo thêm đặc trưng mới, mà còn bao gồm việc lựa chọn và biến đổi đặc trưng một cách hợp lý nhằm tối ưu hóa khả năng học của mô hình. Đây là một bước quan trọng góp phần nâng cao hiệu quả phát hiện gian lận trong điều kiện dữ liệu phức tạp và mất cân bằng.

3.3.3. Train/Val/Test Split

Sau khi hoàn tất các bước làm sạch và xây dựng đặc trưng, dữ liệu được chia thành các tập huấn luyện (train), tập xác thực (validation) và tập kiểm tra (test) nhằm phục vụ cho quá trình huấn luyện, tinh chỉnh và đánh giá mô hình. Việc phân chia dữ liệu một cách hợp lý đóng vai trò quan trọng trong việc đảm bảo tính khách quan và khả năng tổng quát hóa (generalization) của mô hình.

Trong bài toán phát hiện gian lận thẻ tín dụng, dữ liệu có yếu tố thời gian thông qua biến `Time`, do đó đề tài áp dụng phương pháp chia dữ liệu theo thứ tự thời gian (time-based split) thay vì chia ngẫu nhiên. Cụ thể, toàn bộ dữ liệu được sắp xếp tăng dần theo `Time`, sau đó được chia thành các phần theo tỷ lệ nhất định, chẳng hạn 80% đầu tiên được sử dụng cho huấn luyện và 20% còn lại được sử dụng làm tập kiểm tra.

Cách tiếp cận này giúp mô hình học từ dữ liệu trong quá khứ và được đánh giá trên dữ liệu trong tương lai, phù hợp với kịch bản triển khai thực tế.

Tiếp theo, tập huấn luyện ban đầu tiếp tục được chia thành tập train và validation theo tỷ lệ, ví dụ 80/20. Tập train được sử dụng để huấn luyện mô hình, trong khi tập validation đóng vai trò điều chỉnh siêu tham số (hyperparameter tuning), lựa chọn mô hình và tối ưu ngưỡng phân loại (threshold tuning). Việc tách riêng tập validation giúp tránh tình trạng overfitting vào tập test, đảm bảo kết quả đánh giá cuối cùng mang tính khách quan.

Một điểm quan trọng trong quá trình chia dữ liệu là đảm bảo không xảy ra hiện tượng “data leakage”. Trong bối cảnh này, việc chia theo thời gian giúp ngăn chặn việc thông tin từ tương lai bị sử dụng trong quá trình huấn luyện. Đồng thời, toàn bộ quá trình tiền xử lý như scaling hoặc feature engineering (nếu có liên quan đến thống kê toàn cục) cần được thực hiện dựa trên tập train, sau đó áp dụng lại cho tập validation và test để đảm bảo tính nhất quán.

Ngoài ra, do dữ liệu có sự mất cân bằng nghiêm trọng, phân phối giữa các lớp trong từng tập cũng cần được kiểm tra nhằm đảm bảo không có sự sai lệch quá lớn giữa các tập. Điều này giúp quá trình huấn luyện và đánh giá mô hình phản ánh đúng đặc điểm của dữ liệu thực tế.

Tóm lại, việc chia dữ liệu theo chiến lược time-based split kết hợp với phân tách train/validation/test rõ ràng giúp đảm bảo tính thực tiễn, tránh rò rỉ dữ liệu và tạo điều kiện thuận lợi cho việc đánh giá cũng như tối ưu hóa mô hình trong bài toán phát hiện gian lận.

3.4. Xử lý mất cân bằng (Sampling)

Như đã trình bày ở các phần trước, dữ liệu trong bài toán phát hiện gian lận thể hiện sự mất cân bằng nghiêm trọng giữa hai lớp, trong đó lớp giao dịch gian lận (fraud) chiếm tỷ lệ rất nhỏ so với lớp giao dịch hợp lệ (legitimate). Điều này khiến cho các mô hình học máy có xu hướng thiên lệch về phía lớp đa số, dẫn đến khả năng phát hiện gian lận kém nếu không có các biện pháp xử lý phù hợp. Do đó, việc áp dụng các kỹ thuật xử lý mất cân bằng (sampling techniques) là một bước quan trọng trong quy trình xây dựng mô hình.

Một trong những hướng tiếp cận phổ biến là oversampling, trong đó số lượng mẫu của lớp thiểu số được tăng lên nhằm cân bằng phân phối dữ liệu. Phương pháp tiêu biểu trong nhóm này là SMOTE (Synthetic Minority Over-sampling Technique), trong đó các mẫu mới của lớp gian lận được tạo ra bằng cách nội suy giữa các điểm dữ liệu gần nhau trong không gian đặc trưng. Điều này giúp tăng số lượng mẫu gian lận mà không chỉ đơn thuần là sao chép dữ liệu, từ đó giảm nguy cơ overfitting so với các phương pháp oversampling truyền thống.

Ngược lại, undersampling là phương pháp giảm số lượng mẫu của lớp đa số bằng cách loại bỏ một phần các giao dịch hợp lệ. Cách tiếp cận này giúp cân bằng dữ

liệu nhanh chóng và giảm chi phí tính toán, tuy nhiên có thể làm mất đi thông tin quan trọng nếu loại bỏ quá nhiều dữ liệu.

Ngoài ra, các phương pháp kết hợp như SMOTE kết hợp với Tomek Links (SMOTE + Tomek) cũng được sử dụng nhằm vừa tăng cường dữ liệu lớp thiểu số, vừa loại bỏ các điểm dữ liệu nhiễu hoặc nằm gần ranh giới giữa hai lớp. Điều này giúp cải thiện khả năng phân tách của mô hình và giảm thiểu ảnh hưởng của các mẫu không điển hình.

Bên cạnh các kỹ thuật lấy mẫu lại dữ liệu, đề tài cũng xem xét việc điều chỉnh trọng số lớp (class_weight) trong quá trình huấn luyện mô hình. Phương pháp này không thay đổi phân phối dữ liệu mà thay vào đó gán trọng số lớn hơn cho các mẫu thuộc lớp gian lận trong hàm mất mát, từ đó buộc mô hình “quan tâm” nhiều hơn đến lớp thiểu số.

Một điểm quan trọng cần lưu ý là các kỹ thuật sampling chỉ được áp dụng trên tập huấn luyện (train set), không được áp dụng trên tập validation và test nhằm đảm bảo việc đánh giá mô hình phản ánh đúng phân phối dữ liệu thực tế. Đồng thời, việc lựa chọn phương pháp sampling phù hợp cần được kiểm chứng thông qua thực nghiệm, do mỗi phương pháp có thể ảnh hưởng khác nhau đến từng loại mô hình.

Tóm lại, xử lý mất cân bằng thông qua các kỹ thuật sampling và điều chỉnh trọng số lớp là một bước thiết yếu trong bài toán phát hiện gian lận. Việc lựa chọn và kết hợp các phương pháp phù hợp không chỉ giúp cải thiện khả năng phát hiện lớp gian lận mà còn góp phần nâng cao hiệu năng tổng thể của mô hình trong điều kiện dữ liệu phức tạp.

Chương 4. Xây dựng mô hình

4.1. Huấn luyện các mô hình

4.1.1. Logistic Regression

Trong giai đoạn huấn luyện mô hình, Logistic Regression được sử dụng như một mô hình cơ sở (baseline) nhằm thiết lập mốc so sánh với các phương pháp nâng cao như Random Forest hay LightGBM. Mặc dù có cấu trúc đơn giản, Logistic Regression vẫn là một lựa chọn hiệu quả trong các bài toán phân loại nhị phân nhờ khả năng ước lượng xác suất rõ ràng và dễ diễn giải.

Quá trình huấn luyện được thực trên tập train sau khi hoàn tất các bước tiền xử lý và xây dựng đặc trưng. Cụ thể, dữ liệu đầu vào được chuẩn hóa thông qua StandardScaler nhằm đưa các đặc trưng về cùng một thang đo, từ đó giúp quá trình tối ưu hội tụ nhanh và ổn định hơn. Mô hình được triển khai dưới dạng Pipeline kết hợp giữa bước chuẩn hóa và thuật toán học, đảm bảo tính nhất quán trong cả giai đoạn huấn luyện và dự đoán.

Một điểm quan trọng trong bài toán này là dữ liệu có sự mất cân bằng nghiêm trọng giữa hai lớp. Do đó, tham số `class_weight="balanced"` được sử dụng nhằm tự động điều chỉnh trọng số giữa các lớp trong hàm mất mát, giúp mô hình chú trọng hơn đến các mẫu thuộc lớp thiểu số. Cách tiếp cận này góp phần cải thiện khả năng phát hiện gian lận mà không cần thay đổi phân phối dữ liệu ban đầu.

Về nguyên lý, Logistic Regression mô hình hóa xác suất xảy ra gian lận thông qua hàm sigmoid

$$P(y = 1|x) = \frac{1}{1+e^{-(w^T x+b)}} ,$$

trong đó, w là vector trọng số, x là vector đặc trưng đầu vào và b là hệ số bias. Đầu ra của mô hình là xác suất dự đoán cho mỗi giao dịch, từ đó được sử dụng để đánh giá thông qua các chỉ số như ROC-AUC, PR-AUC, Brier Score và các chỉ số calibration.

Kết quả thực nghiệm cho thấy Logistic Regression đạt hiệu năng tương đối tốt trên cả tập validation và test. Cụ thể, trên tập validation, mô hình đạt ROC-AUC ≈ 0.973 và PR-AUC ≈ 0.775 , trong khi trên tập test đạt ROC-AUC ≈ 0.986 và PR-AUC ≈ 0.741 . Các chỉ số calibration như Brier Score và ECE cũng ở mức chấp nhận được, cho thấy xác suất dự đoán có độ tin cậy tương đối.

Do đặc thù của bài toán, việc sử dụng ngưỡng mặc định 0.5 là chưa phù hợp. Thay vào đó, ngưỡng phân loại được tối ưu dựa trên hàm chi phí, trong đó chi phí của việc bỏ sót gian lận cao hơn so với cảnh báo nhầm. Kết quả cho thấy ngưỡng tối ưu trên tập test khoảng 0.969, giúp đạt chi phí thấp nhất và duy trì sự cân bằng hợp lý giữa khả năng phát hiện và tỷ lệ cảnh báo sai.

Ngoài ra, việc điều chỉnh ngưỡng theo mục tiêu Precision cũng được thực hiện. Khi yêu cầu Precision cao (≈ 0.97), mô hình đạt Recall khoảng 0.58; trong khi với mức Precision ≈ 0.90 , Recall tăng lên khoảng 0.67. Điều này cho thấy sự đánh đổi rõ ràng giữa hai chỉ số, đồng thời khẳng định vai trò quan trọng của threshold tuning trong bài toán thực tế.

Một ưu điểm nổi bật của Logistic Regression là tốc độ huấn luyện nhanh, dễ triển khai và không yêu cầu tài nguyên tính toán lớn. Tuy nhiên, do giả định mối quan hệ tuyến tính giữa các đặc trưng và log-odds, mô hình có thể bị hạn chế trong việc nắm bắt các quan hệ phi tuyến phức tạp. Vì vậy, mô hình này chủ yếu đóng vai trò baseline để so sánh với các phương pháp mạnh hơn như LightGBM.

Tổng thể, Logistic Regression cung cấp một nền tảng đơn giản nhưng hiệu quả cho bài toán phát hiện gian lận, đặc biệt khi được kết hợp với các kỹ thuật như chuẩn hóa dữ liệu, điều chỉnh trọng số lớp và tối ưu ngưỡng phân loại.

4.1.2. Random Forest

Huấn luyện mô hình với bộ dữ liệu phát hiện gian lận giao dịch (fraud detection) từ Kaggle, tập huấn luyện đóng vai trò nền tảng để mô hình học mối quan hệ giữa các đặc trưng đầu vào và biến mục tiêu, trong đó mỗi quan sát được gán nhãn là giao dịch hợp lệ hoặc gian lận. Đối với thuật toán Random Forest, quá trình học được thực hiện thông qua việc xây dựng một tập hợp các cây quyết định, trong đó mỗi cây được huấn luyện trên một mẫu bootstrap của dữ liệu gốc. Tại mỗi nút phân tách, thuật toán xem xét ngẫu nhiên một tập con các đặc trưng và lựa chọn đặc trưng tối ưu nhằm tối đa hóa mức độ thuần nhất giữa các lớp (fraud và non-fraud). Cơ chế này giúp mô hình khai thác đa dạng cấu trúc dữ liệu và giảm phương sai trong dự đoán cuối cùng.

4.1.3. XGBoost

Mô hình XGBoost được lựa chọn nhờ những ưu điểm nổi bật. Trước hết, XGBoost là một thuật toán thuộc nhóm boosting, có khả năng kết hợp nhiều cây quyết định yếu (weak learners) để tạo thành một mô hình mạnh, từ đó cải thiện đáng kể độ chính xác. Không giống như các mô hình tuyến tính, XGBoost có khả năng học được các mối quan hệ phi tuyến phức tạp giữa các đặc trưng, điều này đặc biệt phù hợp với bản chất của dữ liệu gian lận.

Bên cạnh đó, XGBoost tích hợp cơ chế regularization (chuẩn hóa), giúp kiểm soát overfitting hiệu quả – một vấn đề thường gặp khi làm việc với dữ liệu lớn và nhiều nhiễu. Ngoài ra, thuật toán này hỗ trợ xử lý dữ liệu thiếu, tối ưu hóa tốc độ tính toán thông qua song song hóa, và cho phép điều chỉnh trọng số lớp (scale_pos_weight), giúp cải thiện hiệu suất trong bối cảnh dữ liệu mất cân bằng.

Quy trình huấn luyện mô hình XGBoost trong bài toán phát hiện gian lận được thực hiện qua nhiều bước, nhằm đảm bảo mô hình đạt hiệu suất cao và khả năng tổng quát tốt.

Đầu tiên, dữ liệu được chia thành các tập huấn luyện (training set), kiểm định (validation set) và kiểm tra (test set). Việc phân chia này giúp đánh giá khách quan khả năng dự đoán của mô hình trên dữ liệu chưa từng thấy. Trong trường hợp dữ liệu bị mất cân bằng, việc chia tập cần đảm bảo giữ nguyên tỷ lệ giữa các lớp.

Tiếp theo, mô hình XGBoost được khởi tạo với các siêu tham số ban đầu. Các tham số quan trọng bao gồm số lượng cây (`n_estimators`), độ sâu của cây (`max_depth`), tốc độ học (`learning_rate`), tỷ lệ lấy mẫu (`subsample`) và trọng số lớp (`scale_pos_weight`). Những tham số này ảnh hưởng trực tiếp đến khả năng học và hiệu suất của mô hình.

Quá trình huấn luyện được thực hiện theo cơ chế lặp, trong đó mỗi vòng lặp sẽ xây dựng một cây mới nhằm giảm thiểu sai số của mô hình hiện tại.

Cuối cùng, mô hình được đánh giá trên tập test thông qua các chỉ số như Precision, Recall, F1-score và AUC-ROC. Trong bài toán phát hiện gian lận, Recall thường được ưu tiên nhằm giảm thiểu việc bỏ sót các trường hợp gian lận.

Về nguyên lý, mô hình hoạt động dựa trên việc xây dựng mô hình theo dạng cộng dồn (additive model). Thay vì xây dựng một mô hình duy nhất, XGBoost kết hợp nhiều cây quyết định theo cách tuần tự, trong đó mỗi cây mới được xây dựng để sửa lỗi của các cây trước đó.

Cụ thể, tại mỗi bước lặp, mô hình sẽ tính toán phần dư (residual) – tức là sai số giữa giá trị dự đoán và giá trị thực tế. Sau đó, một cây quyết định mới được huấn luyện để dự đoán phần dư này. Kết quả của cây mới sẽ được cộng vào mô hình hiện tại với một hệ số điều chỉnh (`learning_rate`), giúp mô hình dần dần cải thiện qua từng vòng lặp.

Điểm khác biệt quan trọng của XGBoost so với các phương pháp boosting truyền thống là việc sử dụng thông tin gradient và hessian (đạo hàm bậc nhất và bậc hai của hàm mất mát) để tối ưu hóa hàm mục tiêu. Nhờ đó, quá trình học trở nên chính xác và hiệu quả hơn.

Ngoài ra, XGBoost sử dụng chiến lược cắt tia cây (tree pruning), xử lý dữ liệu thưa (sparse data) và tối ưu hóa bộ nhớ, giúp tăng tốc độ huấn luyện và giảm chi phí tính toán.

4.1.4. LightGBM

Trong giai đoạn huấn luyện mô hình, LightGBM được lựa chọn như một trong những thuật toán trọng tâm nhờ khả năng xử lý hiệu quả dữ liệu có kích thước lớn, tốc độ huấn luyện nhanh và năng lực học tốt trên các quan hệ phi tuyến giữa đặc trưng và biến mục tiêu. Đối với bài toán phát hiện gian lận thẻ tín dụng, đây là một lựa chọn phù hợp vì dữ liệu có tính mất cân bằng cao, đồng thời tồn tại nhiều đặc trưng đã được biến đổi như nhóm biến V1–V28, kết hợp với các đặc trưng bổ sung được xây dựng từ Amount và Time.

Quá trình huấn luyện LightGBM được thực hiện trên tập train sau khi hoàn tất các bước tiền xử lý và xây dựng đặc trưng. Do bài toán có tỷ lệ gian lận rất thấp, mô hình không được tối ưu theo tiêu chí accuracy thông thường mà tập trung vào khả năng phát hiện lớp thiểu số. Vì vậy, trong quá trình huấn luyện, các tham số của mô hình được lựa chọn theo hướng vừa đảm bảo khả năng học được các mẫu gian lận, vừa hạn chế hiện tượng overfitting. Các siêu tham số quan trọng bao gồm số lượng cây quyết định (`n_estimators`), tốc độ học (`learning_rate`), số lá tối đa trên mỗi cây (`num_leaves`), số mẫu tối thiểu tại nút lá (`min_child_samples`), cùng với các tham số lấy mẫu theo hàng và cột như `subsample` và `colsample_bytree`.

Về nguyên lý, LightGBM xây dựng mô hình cộng dồn từ nhiều cây quyết định yếu. Ở bước lặp thứ t , mô hình được cập nhật theo dạng

$$\hat{y}^t = \hat{y}^{(t-1)} + \eta f_t(x),$$

trong đó, $\hat{y}^t$ là đầu ra của mô hình tại vòng lặp thứ t , η là tốc độ học (`learning_rate`), và $f_t(x)$ là cây quyết định được thêm vào ở vòng lặp hiện tại. Mục tiêu của từng cây mới là giảm phần sai số còn lại của mô hình trước đó thông qua việc tối ưu hàm mất mát.

Trong bài toán phân loại nhị phân, LightGBM thường xuất ra xác suất dự đoán thông qua hàm sigmoid

$$P(y = 1|x) = \frac{1}{1+e^{-z(x)}},$$

trong đó, $z(x)$ là tổng điểm số mà toàn bộ các cây tạo ra cho mẫu x . Xác suất này sau đó được dùng để đánh giá mô hình theo các chỉ số như PR-AUC, ROC-AUC, Brier Score và các chỉ số calibration.

Để thích nghi với đặc điểm mất cân bằng dữ liệu, quá trình huấn luyện LightGBM có thể kết hợp cơ chế gán trọng số lớp, giúp tăng mức độ ảnh hưởng của

lớp gian lận trong hàm mất mát. Điều này đặc biệt cần thiết vì nếu huấn luyện theo phân phối gốc, mô hình rất dễ nghiêng về lớp giao dịch hợp lệ. Bên cạnh đó, mô hình được đánh giá trên tập validation nhằm lựa chọn bộ siêu tham số phù hợp nhất trước khi áp dụng lên tập test.

Một ưu điểm nổi bật của LightGBM trong đề tài này là khả năng khai thác tốt các đặc trưng đã được biến đổi mà không đòi hỏi chuẩn hóa dữ liệu nghiêm ngặt như một số mô hình tuyến tính. Hơn nữa, do cơ chế phát triển cây theo hướng leaf-wise, LightGBM có thể tìm được các điểm chia dữ liệu mang tính phân biệt cao, từ đó nâng cao hiệu quả nhận diện các giao dịch bất thường. Tuy nhiên, chính cơ chế này cũng khiến mô hình dễ overfit nếu số lá quá lớn hoặc độ sâu cây không được kiểm soát hợp lý.

Sau khi hoàn tất quá trình huấn luyện, mô hình LightGBM không chỉ được đánh giá ở mức độ phân biệt giữa hai lớp mà còn tiếp tục được xem xét ở khía cạnh hiệu chỉnh xác suất và tối ưu ngưỡng phân loại. Điều này là cần thiết vì trong bài toán phát hiện gian lận, một mô hình có xác suất dự đoán tốt sẽ hỗ trợ hiệu quả hơn cho bước ra quyết định vận hành, đặc biệt khi cân cân bằng giữa khả năng phát hiện gian lận và số lượng cảnh báo sai.

4.2. Đánh giá mô hình

4.3. Tối ưu tham số

4.3.1. Tối ưu tham số Logistic Regression

Sau khi xây dựng mô hình Logistic Regression cơ bản, bước tiếp theo là tiến hành tối ưu hóa nhằm nâng cao hiệu năng mô hình, đặc biệt trong bối cảnh dữ liệu mất cân bằng nghiêm trọng của bài toán phát hiện gian lận.

Trong quá trình huấn luyện, thay vì chỉ sử dụng tham số `class_weight`, mô hình được kết hợp với các kỹ thuật sampling nhằm cải thiện khả năng học của lớp thiểu số (giao dịch gian lận). Cụ thể, các phương pháp được xem xét bao gồm: không xử lý mất cân bằng, Random UnderSampling (giảm số lượng mẫu của lớp đa số), SMOTE (tạo mẫu tổng hợp cho lớp thiểu số), và SMOTE kết hợp Tomek (vừa oversampling vừa loại bỏ nhiễu).

Quá trình huấn luyện được thực hiện thông qua Pipeline của thư viện `imbalanced-learn`, đảm bảo các bước sampling chỉ được áp dụng trên tập train, từ đó tránh hiện tượng rò rỉ dữ liệu (data leakage).

Kết quả thực nghiệm cho thấy sự khác biệt rõ rệt giữa các phương pháp. Khi không sử dụng sampling, mô hình đạt AUC-PR khoảng 0.71 nhưng Recall thấp, chỉ khoảng 0.51. Phương pháp UnderSampling giúp tăng Recall lên khoảng 0.90, tuy nhiên lại làm giảm đáng kể AUC-PR. Trong khi đó, SMOTE và SMOTE kết hợp Tomek cho hiệu năng tốt nhất, với AUC-PR đạt khoảng 0.77, Recall khoảng 0.89 và ROC-AUC khoảng 0.96. Đồng thời, xét theo hàm chi phí tại ngưỡng mặc định 0.5, hai phương pháp này cũng cho chi phí thấp nhất, khoảng 6090, tốt hơn so với các phương pháp còn lại.

Từ các kết quả trên, phương pháp SMOTE kết hợp Tomek được lựa chọn làm kỹ thuật xử lý mất cân bằng chính cho mô hình Logistic Regression trong các bước tiếp theo.

Bên cạnh đó, do Logistic Regression đưa ra xác suất dự đoán, việc sử dụng ngưỡng mặc định 0.5 là không phù hợp trong bài toán mất cân bằng. Vì vậy, ngưỡng phân loại được tối ưu dựa trên hàm chi phí (cost-sensitive). Quá trình này được hiển thị bằng cách quét nhiều giá trị threshold trong khoảng từ 0 đến 1, tính toán chi phí tương ứng tại mỗi giá trị, và lựa chọn ngưỡng cho chi phí nhỏ nhất.

Kết quả cho thấy ngưỡng tối ưu đạt khoảng 0.945, với chi phí tối thiểu là 2700, giảm đáng kể so với mức 6090 khi sử dụng ngưỡng mặc định 0.5. Tại ngưỡng này, mô hình đạt True Positive (TP) là 63, False Positive (FP) là 100, False Negative (FN) là 11 và True Negative (TN) là 56572. Điều này cho thấy mô hình đã cải thiện đáng kể khả năng phát hiện gian lận thông qua việc giảm số lượng FN, đồng thời vẫn kiểm soát được số lượng cảnh báo sai ở mức hợp lý.

Tổng thể, việc kết hợp Logistic Regression với các kỹ thuật sampling và tối ưu ngưỡng phân loại đã mang lại cải thiện rõ rệt so với mô hình ban đầu. Cụ thể, sampling (đặc biệt là SMOTE kết hợp Tomek) giúp mô hình học tốt hơn đối với lớp thiểu số, trong khi threshold tuning giúp chuyển đổi xác suất dự đoán thành quyết định phân loại phù hợp với mục tiêu tối ưu chi phí. Nhờ đó, hiệu năng mô hình được cải thiện cả về Recall, AUC-PR và chi phí tổng thể.

Tuy nhiên, do bản chất là mô hình tuyến tính, Logistic Regression vẫn tồn tại hạn chế trong việc nắm bắt các quan hệ phi tuyến phức tạp trong dữ liệu. Vì vậy, mô hình này được xem như một baseline mạnh đã được tối ưu, đóng vai trò làm cơ sở so sánh với các mô hình nâng cao như LightGBM và XGBoost.

4.3.2. Tối ưu tham số Random Forest

Sau khi huấn luyện mô hình ban đầu, tôi tiến hành tối ưu siêu tham số nhằm cải thiện hiệu suất dự đoán của mô hình Random Forest bằng phương pháp Random Search. Thay vì thử toàn bộ các tổ hợp tham số như Grid Search, Random Search thực hiện việc lấy ngẫu nhiên các giá trị trong không gian siêu tham số đã xác định trước. Cụ thể, tôi thiết lập các khoảng giá trị cho các tham số như số lượng cây (`n_estimators`), độ sâu tối đa của cây (`max_depth`), và số lượng đặc trưng được chọn tại mỗi nút (`max_features`), sau đó tiến hành lặp qua nhiều vòng thử nghiệm, mỗi vòng chọn ngẫu nhiên một bộ tham số để huấn luyện và đánh giá trên tập validation.

Dựa trên kết quả đánh giá, bộ siêu tham số mang lại hiệu suất tốt nhất được lựa chọn để xây dựng mô hình cuối cùng. Cách tiếp cận này giúp giảm đáng kể chi phí tính toán so với Grid Search, đồng thời vẫn đảm bảo khả năng tìm kiếm được cấu hình tham số tối ưu hoặc gần tối ưu, từ đó nâng cao hiệu quả và khả năng tổng quát hóa của mô hình trong bài toán phát hiện gian lận giao dịch.

4.3.3. Tối ưu tham số XGBoost

Hiệu suất của mô hình XGBoost phụ thuộc rất lớn vào việc lựa chọn và điều chỉnh các siêu tham số (hyperparameters). Trong bài toán phát hiện gian lận, nơi dữ liệu có tính chất phức tạp và mất cân bằng, việc tối ưu tham số đóng vai trò quan trọng nhằm nâng cao khả năng dự đoán và giảm thiểu hiện tượng overfitting.

Trước hết, các siêu tham số của XGBoost có thể được chia thành ba nhóm chính: nhóm điều khiển độ phức tạp của mô hình, nhóm điều khiển quá trình học và nhóm xử lý dữ liệu.

Nhóm tham số kiểm soát độ phức tạp bao gồm `max_depth`, `min_child_weight` và `gamma`. Tham số `max_depth` quy định độ sâu tối đa của mỗi cây, ảnh hưởng trực tiếp đến khả năng học các mối quan hệ phi tuyến. Giá trị lớn giúp mô hình học tốt hơn nhưng dễ gây overfitting. Tham số `min_child_weight` kiểm soát số lượng mẫu tối thiểu tại một node, giúp hạn chế việc chia tách không cần thiết. Trong khi đó, `gamma` xác định mức độ giảm loss tối thiểu để thực hiện một phép chia node, từ đó giúp mô hình trở nên bảo thủ hơn trong việc mở rộng cây.

Nhóm tham số điều khiển quá trình học bao gồm `learning_rate` (η) và `n_estimators`. `learning_rate` quyết định mức độ đóng góp của mỗi cây vào mô hình tổng thể; giá trị nhỏ giúp mô hình học chậm nhưng ổn định hơn. Ngược lại,

`n_estimators` là số lượng cây được xây dựng; hai tham số này thường có mối quan hệ đánh đổi với nhau. Ngoài ra, các tham số như `subsample` và `colsample_bytree` được sử dụng để lấy mẫu ngẫu nhiên dữ liệu và đặc trưng, góp phần giảm overfitting và tăng khả năng tổng quát hóa.

Đối với bài toán phát hiện gian lận, tham số `scale_pos_weight` đặc biệt quan trọng do dữ liệu thường bị mất cân bằng nghiêm trọng. Việc thiết lập tham số này giúp tăng trọng số cho lớp thiểu số (gian lận), từ đó cải thiện khả năng phát hiện các trường hợp gian lận hiếm gặp.

Quá trình tối ưu tham số thường được thực hiện thông qua các phương pháp như Grid Search hoặc Random Search kết hợp với cross-validation. Trong nghiên cứu này, các tập giá trị của từng tham số được xác định trước, sau đó mô hình được huấn luyện và đánh giá trên tập validation để lựa chọn cấu hình tối ưu. Bên cạnh đó, kỹ thuật early stopping cũng được áp dụng nhằm dừng quá trình huấn luyện khi hiệu suất không còn cải thiện, giúp tránh overfitting và tiết kiệm tài nguyên tính toán.

Việc tối ưu tham số không chỉ giúp cải thiện độ chính xác mà còn nâng cao các chỉ số quan trọng trong bài toán phát hiện gian lận như Recall, F1-score và AUC-ROC. Do đặc thù của bài toán, quá trình tối ưu thường ưu tiên các cấu hình giúp tăng Recall nhằm giảm thiểu việc bỏ sót các giao dịch gian lận.

4.3.4. Tối ưu tham số LightGBM

Sau khi xây dựng và huấn luyện mô hình LightGBM, bước tiếp theo là tiến hành tối ưu hóa siêu tham số (hyperparameter tuning) nhằm nâng cao hiệu năng mô hình và đảm bảo khả năng tổng quát hóa trên dữ liệu chưa từng thấy. Do đặc điểm của LightGBM là một mô hình mạnh nhưng nhạy cảm với cấu hình tham số, việc lựa chọn bộ siêu tham số phù hợp có ảnh hưởng trực tiếp đến chất lượng dự đoán, đặc biệt trong bối cảnh dữ liệu mất cân bằng như bài toán phát hiện gian lận.

Quá trình tối ưu tham số được thực hiện trên tập validation nhằm tránh overfitting vào tập test. Thay vì tối ưu theo các chỉ số truyền thống như Accuracy, đề tài tập trung vào các thước đo phù hợp với bài toán như AUC-PR, Recall và đồng thời xem xét các chỉ số calibration như Brier Score và ECE. Điều này đảm bảo rằng mô hình không chỉ phân biệt tốt giữa hai lớp mà còn cung cấp xác suất dự đoán đáng tin cậy.

Các siêu tham số quan trọng của LightGBM được xem xét trong quá trình tối ưu bao gồm số lượng cây quyết định (`n_estimators`), tốc độ học (`learning_rate`), số lượng lá tối đa (`num_leaves`), số mẫu tối thiểu tại mỗi nút lá (`min_child_samples`), cũng như các tham số liên quan đến regularization như `lambda_1` và `lambda_2` và các tham số lấy mẫu như `subsample` và `colsample_bytree`. Việc điều chỉnh các tham số này nhằm đạt được sự cân bằng giữa khả năng học biểu diễn phức tạp của dữ liệu và việc hạn chế overfitting.

Về phương pháp, việc tối ưu tham số có thể được thực hiện thông qua các kỹ thuật như Grid Search hoặc Random Search, trong đó Random Search thường được ưu tiên do khả năng khám phá không gian tham số hiệu quả hơn với chi phí tính toán hợp lý. Ngoài ra, việc theo dõi hiệu năng mô hình trên tập validation trong quá trình huấn luyện (early stopping) cũng được áp dụng nhằm dừng huấn luyện khi mô hình không còn cải thiện, từ đó tránh overfitting và giảm thời gian tính toán.

Một điểm quan trọng trong đề tài là việc kết hợp tối ưu tham số với xử lý mất cân bằng dữ liệu. Cụ thể, các cấu hình tham số được đánh giá trong bối cảnh đã áp dụng `sampling` hoặc `class_weight`, nhằm tìm ra bộ tham số phù hợp nhất với từng chiến lược xử lý dữ liệu. Điều này giúp đảm bảo rằng kết quả tối ưu không bị lệ thuộc vào một cấu hình dữ liệu cụ thể.

Bên cạnh đó, quá trình tối ưu không chỉ dừng lại ở việc lựa chọn tham số cho mô hình, mà còn mở rộng sang việc tối ưu ngưỡng phân loại (threshold tuning). Sau khi mô hình được huấn luyện với bộ tham số tối ưu, các xác suất dự đoán trên tập validation được sử dụng để quét qua nhiều ngưỡng khác nhau, từ đó lựa chọn ngưỡng phù hợp nhất dựa trên tiêu chí như tối đa hóa F1-score, đạt $\text{Precision} \geq 90\%$ hoặc tối thiểu hóa chi phí sai lầm. Đây là một bước quan trọng giúp chuyển đổi đầu ra xác suất của mô hình thành quyết định thực tế.

Tóm lại, quá trình tối ưu tham số LightGBM trong đề tài được thực hiện một cách có hệ thống, kết hợp giữa lựa chọn siêu tham số, theo dõi hiệu năng trên tập validation và tối ưu ngưỡng phân loại. Sự kết hợp này giúp khai thác tối đa tiềm năng của LightGBM, đồng thời đảm bảo mô hình đạt hiệu năng cao và phù hợp với yêu cầu thực tế của bài toán phát hiện gian lận.

4.4. Ensemble (kết hợp mô hình)

4.4.1. LightGBM + Random Forest

Nhằm nâng cao hiệu quả dự đoán cho bài toán phát hiện gian lận thẻ tín dụng, phương pháp kết hợp mô hình được áp dụng thông qua việc tích hợp hai thuật toán học máy phổ biến là LightGBM và Random Forest. Cách tiếp cận này dựa trên nguyên lý của học tổ hợp (ensemble learning), trong đó nhiều mô hình được phối hợp để cải thiện độ chính xác và khả năng tổng quát hóa so với từng mô hình riêng lẻ.

Hai mô hình được lựa chọn do sở hữu những đặc điểm bổ trợ lẫn nhau. LightGBM hoạt động theo cơ chế boosting, xây dựng các cây quyết định theo hướng tuần tự, trong đó mỗi cây mới được huấn luyện nhằm giảm thiểu sai số còn lại của các cây trước. Nhờ vậy, mô hình có khả năng học tốt các quan hệ phi tuyến phức tạp và đặc biệt hiệu quả trong việc nhận diện các mẫu dữ liệu hiếm như giao dịch gian lận. Trong khi đó, Random Forest lại dựa trên cơ chế bagging, tạo ra nhiều cây quyết định độc lập từ các tập con dữ liệu khác nhau và tổng hợp kết quả dự đoán. Cách tiếp cận này giúp giảm phương sai và tăng độ ổn định của mô hình.

Quy trình kết hợp được thực hiện bằng cách huấn luyện độc lập hai mô hình trên cùng một tập dữ liệu đã được tiền xử lý và cân bằng. Sau khi huấn luyện, mỗi mô hình sẽ đưa ra xác suất dự đoán cho từng mẫu dữ liệu. Các xác suất này sau đó được tổng hợp thông qua phương pháp trung bình có trọng số, trong đó đầu ra cuối cùng là sự kết hợp tuyến tính giữa dự đoán của hai mô hình. Trọng số của từng mô hình được xác định thông qua quá trình đánh giá trên tập validation nhằm tối ưu các chỉ số đo lường hiệu năng như ROC-AUC hoặc PR-AUC.

Việc kết hợp LightGBM và Random Forest không chỉ giúp tận dụng ưu điểm của từng thuật toán mà còn góp phần giảm thiểu các hạn chế riêng lẻ. LightGBM có thể đạt hiệu năng cao nhưng dễ bị overfitting nếu không được kiểm soát tốt, trong khi Random Forest lại có xu hướng ổn định nhưng đôi khi chưa đủ nhạy để phát hiện các mẫu hiếm. Sự kết hợp giữa hai mô hình vì vậy mang lại sự cân bằng giữa độ chính xác và khả năng tổng quát hóa, đặc biệt phù hợp với các bài toán có dữ liệu mất cân bằng như phát hiện gian lận.

Tuy nhiên, việc triển khai mô hình kết hợp cũng làm gia tăng chi phí tính toán và độ phức tạp của hệ thống. Do đó, trong thực tế, cần cân nhắc giữa lợi ích về hiệu năng và yêu cầu về tài nguyên để lựa chọn phương án phù hợp.

4.4.2. XGBoost + Random Forest

Trong nghiên cứu này, tôi đã tiến hành xây dựng mô hình ensemble bằng cách kết hợp hai thuật toán học máy mạnh là Random Forest và XGBoost nhằm khai thác đồng thời ưu điểm của cả phương pháp bagging và boosting. Random Forest đóng vai trò giảm phương sai thông qua việc tổng hợp dự đoán từ nhiều cây quyết định độc lập, trong khi XGBoost tập trung vào việc tối ưu sai số còn lại của các mô hình trước đó thông qua cơ chế học tuần tự. Sự kết hợp này giúp mô hình có khả năng vừa ổn định vừa nhạy với các mẫu dữ liệu phức tạp, đặc biệt phù hợp với bài toán phát hiện gian lận giao dịch.

Sau khi huấn luyện riêng lẻ từng mô hình, tôi tiến hành xây dựng mô hình ensemble bằng cách tổng hợp đầu ra của Random Forest và XGBoost thông qua cơ chế voting dựa trên xác suất (soft voting). Cụ thể, xác suất dự đoán của hai mô hình được trung bình hóa để đưa ra kết quả cuối cùng. Cách tiếp cận này giúp giảm thiểu sai lệch của từng mô hình đơn lẻ, đồng thời tận dụng được tính đa dạng trong cách học của hai thuật toán, từ đó nâng cao độ tin cậy của dự đoán.

Kết quả thực nghiệm cho thấy mô hình ensemble đạt hiệu suất tốt hơn so với từng mô hình riêng lẻ, thể hiện qua việc cải thiện các chỉ số đánh giá như độ chính xác và khả năng phát hiện lớp gian lận. Điều này chứng minh rằng việc kết hợp Random Forest và XGBoost giúp tăng cường khả năng tổng quát hóa của mô hình, giảm hiện tượng overfitting và mang lại hiệu quả dự đoán ổn định hơn trong bài toán phát hiện gian lận giao dịch.

4.4.3. LightGBM + XGBoost

Bên cạnh việc sử dụng các mô hình đơn lẻ, đề tài tiếp tục khai thác kỹ thuật ensemble nhằm kết hợp ưu điểm của nhiều mô hình khác nhau, từ đó nâng cao hiệu năng tổng thể. Trong đó, việc kết hợp giữa LightGBM và XGBoost được lựa chọn do cả hai đều là các mô hình boosting mạnh, nhưng có sự khác biệt trong cách xây dựng cây và tối ưu hóa, dẫn đến khả năng bổ sung lẫn nhau trong quá trình dự đoán.

Cụ thể, LightGBM sử dụng chiến lược phát triển cây theo hướng leaf-wise, giúp tìm kiếm các điểm chia tối ưu nhanh chóng và hiệu quả, trong khi XGBoost thường sử dụng cơ chế level-wise, mang lại tính ổn định và kiểm soát overfitting tốt hơn. Sự khác biệt này giúp hai mô hình học được các biểu diễn khác nhau của dữ liệu, từ đó tạo ra tính đa dạng (diversity) – yếu tố quan trọng trong các phương pháp ensemble.

Trong đề tài, phương pháp ensemble được sử dụng là kết hợp tuyến tính (weighted averaging) trên xác suất dự đoán của hai mô hình. Cụ thể, xác suất dự đoán cuối cùng được tính theo công thức

$$p_{\text{ensemble}} = w_1 \cdot p_{\text{LGBM}} + w_2 \cdot p_{\text{XGB}},$$

trong đó p_{LGBM} và p_{XGB} lần lượt là xác suất dự đoán của LightGBM và XGBoost, còn w_1, w_2 là các trọng số tương ứng với từng mô hình, thỏa mãn $w_1 + w_2 = 1$. Việc lựa chọn các trọng số này được thực hiện thông qua quá trình thực nghiệm trên tập validation, với mục tiêu tối ưu các chỉ số như AUC-PR, F1-score hoặc chi phí sai lầm.

Quá trình huấn luyện ensemble được thực hiện theo hai bước chính. Trước hết, hai mô hình LightGBM và XGBoost được huấn luyện độc lập trên cùng một tập dữ liệu đã được tiền xử lý. Sau đó, xác suất dự đoán của từng mô hình trên tập validation được sử dụng để tìm ra tổ hợp trọng số tối ưu. Việc này có thể được thực hiện bằng cách thử nghiệm nhiều giá trị khác nhau (ví dụ: từ 0.1 đến 0.9) và lựa chọn cấu hình mang lại hiệu năng tốt nhất theo tiêu chí đã đề ra.

Một điểm quan trọng trong đề tài là việc đánh giá ensemble không chỉ dựa trên khả năng phân loại mà còn xem xét đến độ hiệu chỉnh xác suất (calibration). Trong một số trường hợp, mặc dù ensemble giúp cải thiện AUC-PR, nhưng lại có thể làm giảm chất lượng calibration. Do đó, các chỉ số như Brier Score và ECE cũng được sử dụng để đánh giá toàn diện hiệu năng của mô hình kết hợp.

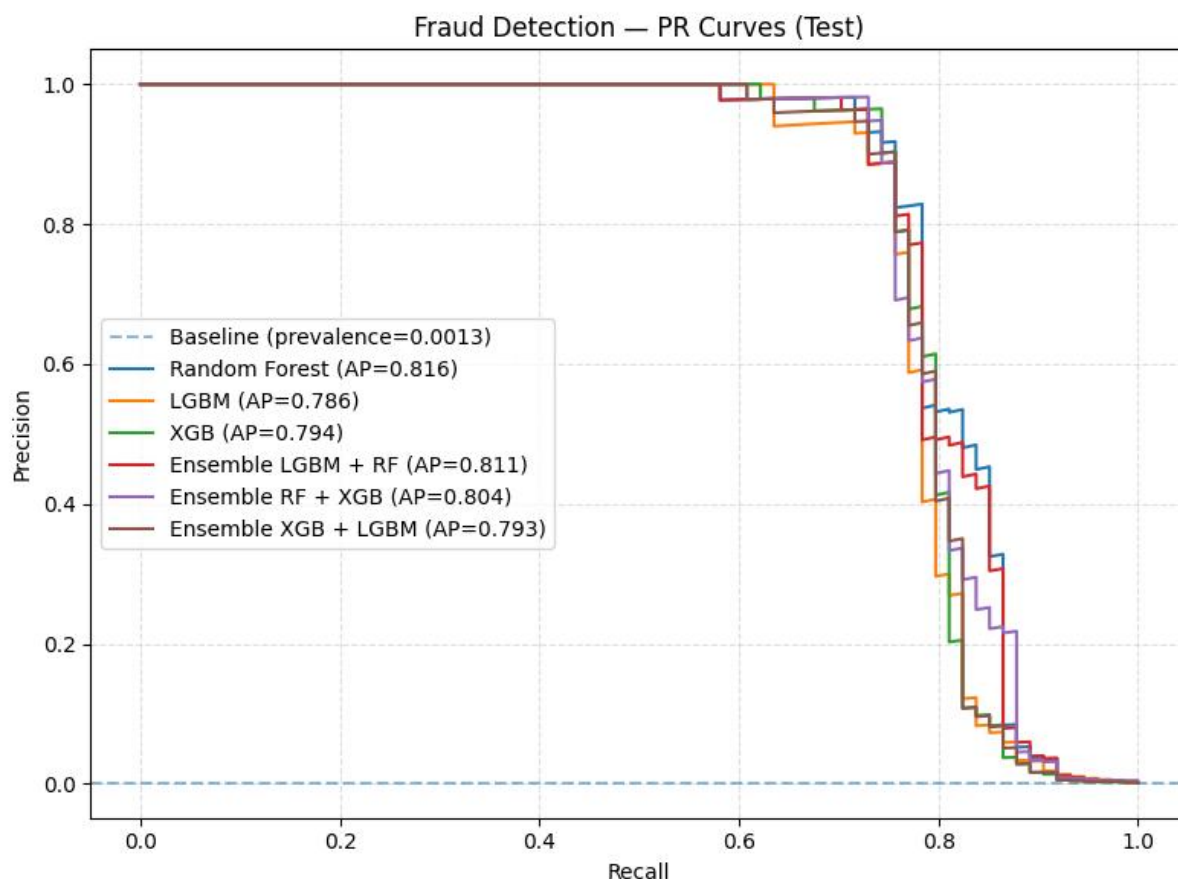
Ngoài ra, tương tự như các mô hình đơn lẻ, đầu ra của ensemble cũng cần được tối ưu ngưỡng phân loại (threshold tuning) để phù hợp với mục tiêu thực tế, chẳng hạn như đạt Precision $\geq 90\%$ hoặc tối thiểu hóa chi phí sai lầm. Việc tối ưu ngưỡng sau ensemble thường mang lại hiệu quả cao hơn so với từng mô hình riêng lẻ, do xác suất dự đoán đã được “làm mượt” thông qua quá trình kết hợp.

Tóm lại, phương pháp ensemble giữa LightGBM và XGBoost giúp tận dụng được thế mạnh của cả hai mô hình, đồng thời cải thiện khả năng tổng quát hóa và độ ổn định trong dự đoán. Đây là một bước nâng cao quan trọng trong đề tài, góp phần đạt được hiệu năng tốt hơn trong bài toán phát hiện gian lận với dữ liệu mất cân bằng.

4.5. Phân tích Precision-Recall Curve

Precision-Recall Curve (PR Curve) là công cụ quan trọng trong việc đánh giá mô hình đối với bài toán phát hiện gian lận thể tín dụng, đặc biệt trong bối cảnh dữ

liệu mất cân bằng. Không giống như ROC Curve, PR Curve tập trung trực tiếp vào hiệu năng của lớp thiểu số (fraud), do đó phản ánh chính xác hơn khả năng phát hiện gian lận của mô hình.



Hình 13: Fraud Detection - PR Curves (Test)

từ biểu đồ, có thể nhận thấy đường baseline (prevalence ≈ 0.0013) nằm ở mức rất thấp, cho thấy nếu dự đoán ngẫu nhiên thì Precision gần như bằng tỷ lệ xuất hiện của gian lận trong dữ liệu. Việc tất cả các mô hình đều có đường cong nằm cao hơn đáng kể so với baseline chứng tỏ chúng đã học được các đặc trưng phân biệt có ý nghĩa.

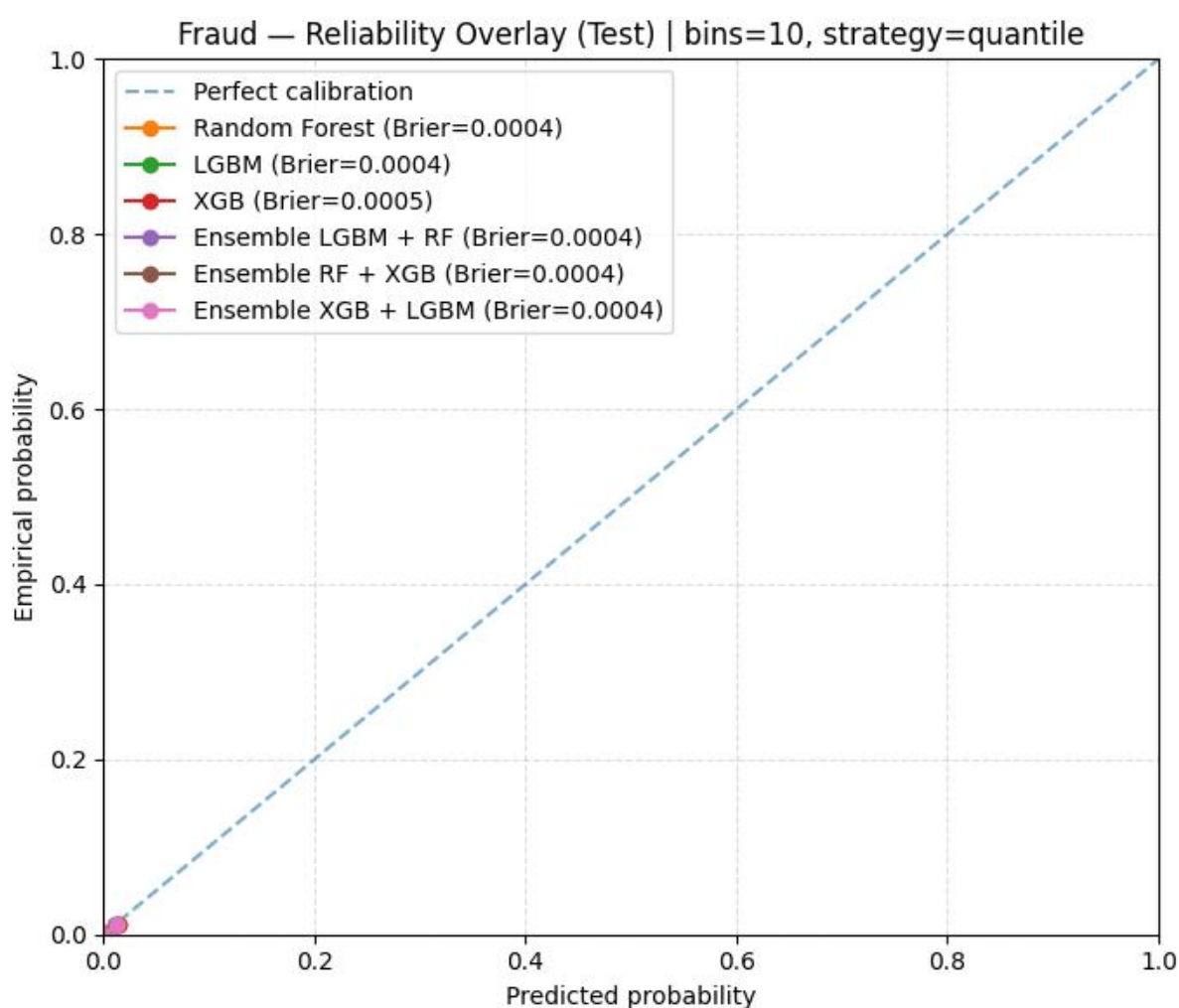
Xét về hiệu năng tổng thể, Random Forest là mô hình tốt nhất với AUC-PR (AP) ≈ 0.816 , cho thấy khả năng duy trì Precision cao trên nhiều mức Recall. Các mô hình boosting như LightGBM (AP ≈ 0.786) và XGBoost (AP ≈ 0.794) có hiệu năng thấp hơn một chút, tuy nhiên vẫn thể hiện khả năng học tốt các mối quan hệ phi tuyến trong dữ liệu.

Đối với các mô hình ensemble, tổ hợp LightGBM + Random Forest đạt AP ≈ 0.811 , gần tiệm cận Random Forest đơn lẻ, cho thấy việc kết hợp giúp cải thiện so với LightGBM và tăng tính ổn định. Ngược lại, các tổ hợp như Random Forest + XGBoost

(AP $\approx$ 0.804) và XGBoost + LightGBM (AP $\approx$ 0.793) không mang lại cải thiện rõ rệt, cho thấy hiệu quả của ensemble phụ thuộc mạnh vào mức độ đa dạng giữa các mô hình thành phần.

Một đặc điểm quan trọng quan sát được từ PR Curve là ở vùng Recall thấp đến trung bình (khoảng 0.6–0.7), hầu hết các mô hình đều đạt Precision gần bằng 1.0, cho thấy khả năng phát hiện các giao dịch gian lận “rõ ràng” là rất tốt. Tuy nhiên, khi Recall tăng cao (trên 0.8), Precision giảm nhanh, phản ánh sự đánh đổi tất yếu giữa việc phát hiện nhiều gian lận hơn và việc gia tăng số lượng cảnh báo sai.

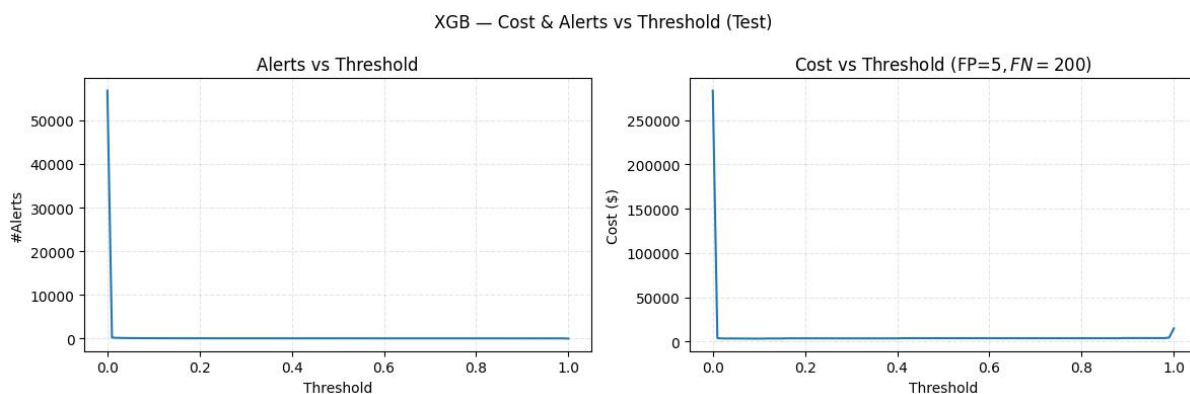
Bên cạnh khả năng phân loại, độ hiệu chỉnh xác suất của mô hình cũng được đánh giá thông qua Reliability Curve.



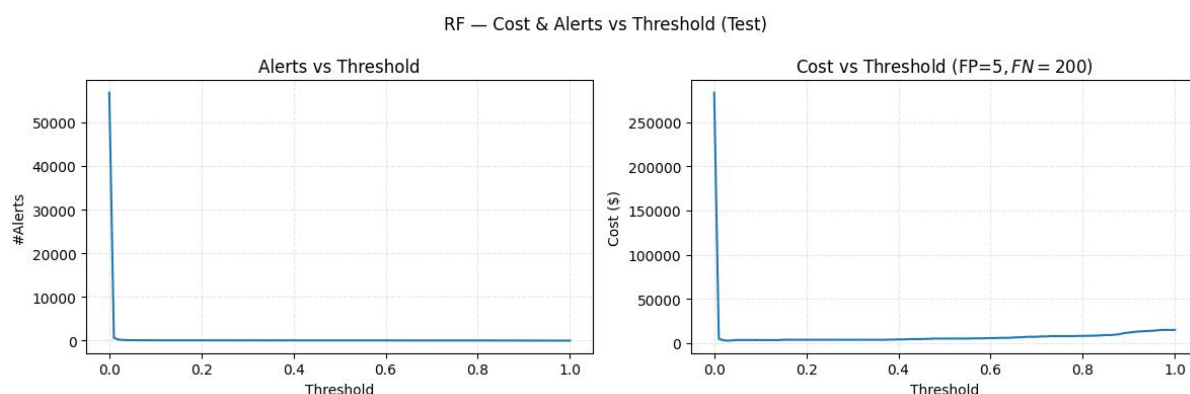
Hình 14: Fraud - Reliability Overlay (Test) | bins=10, strategy=quantile từ biểu đồ, có thể thấy các mô hình đều nằm khá gần đường chéo lý tưởng, cho thấy xác suất dự đoán tương đối đáng tin cậy. Giá trị Brier Score của các mô hình đều rất thấp (khoảng 0.0004–0.0005), chứng tỏ sai số giữa xác suất dự đoán và thực tế là nhỏ.

Điều này có ý nghĩa quan trọng trong việc thiết lập threshold và tối ưu hóa quyết định trong thực tế.

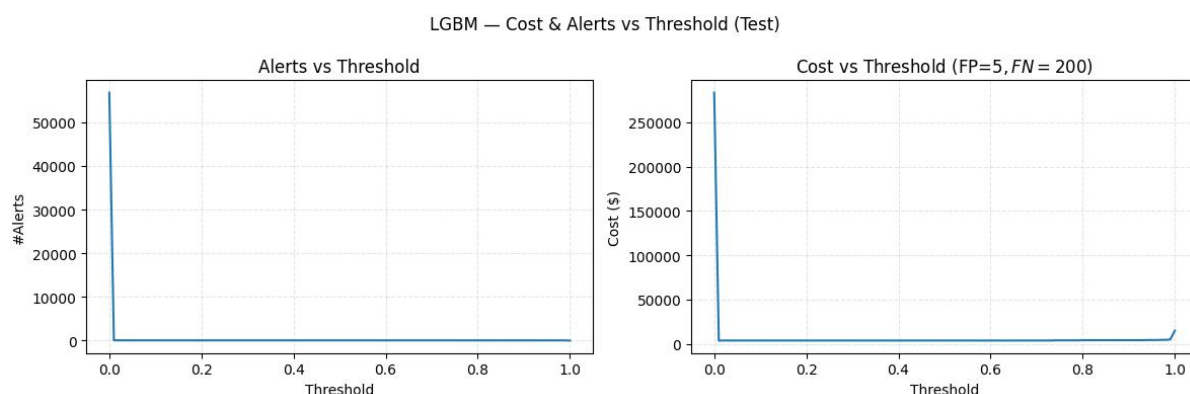
Để đánh giá sâu hơn khả năng vận hành của mô hình, mối quan hệ giữa threshold, số lượng cảnh báo (alerts) và chi phí cũng được phân tích.



Hình 15: XGB - Cost & Alerts vs Threshold (Test)



Hình 16: RF - Cost & Alerts vs Threshold (Test)

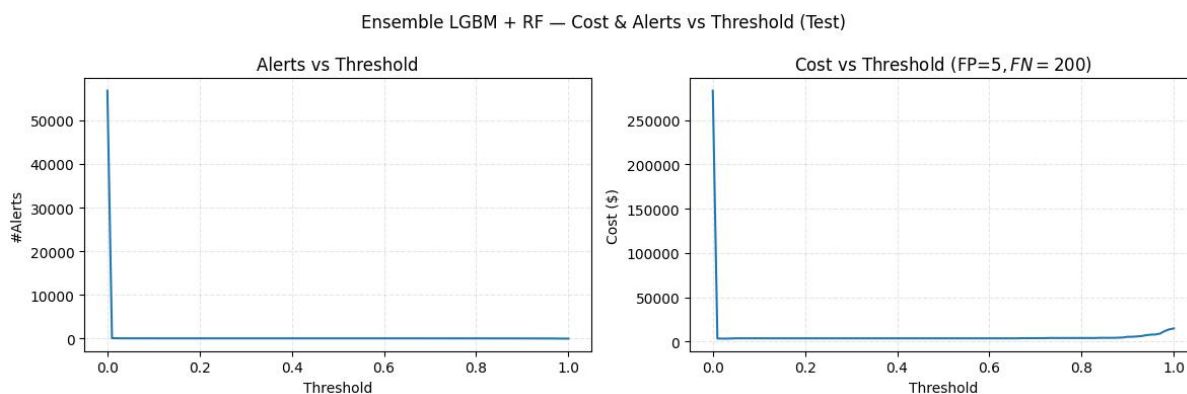


Hình 17: LGBM - Cost & Alerts vs Threshold (Test)

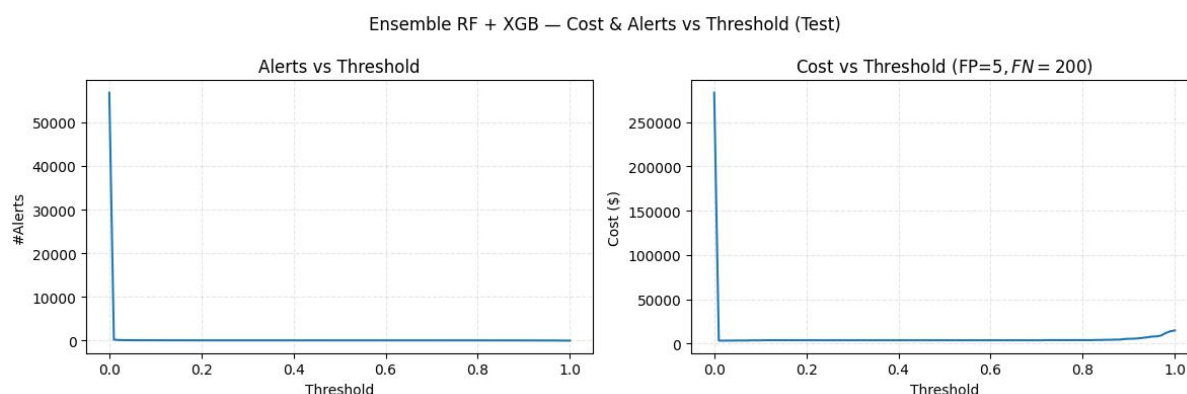
từ các biểu đồ này, có thể thấy rằng khi threshold rất thấp, số lượng cảnh báo tăng mạnh, kéo theo chi phí lớn do số lượng False Positive cao. Khi threshold tăng lên, số cảnh báo giảm nhanh và chi phí giảm đáng kể. Tuy nhiên, nếu threshold quá cao, chi phí lại tăng trở lại do số lượng False Negative tăng, tức là bỏ sót các giao dịch gian lận.

Điều này cho thấy tồn tại một ngưỡng tối ưu cần được lựa chọn để cân bằng giữa hai loại sai lầm.

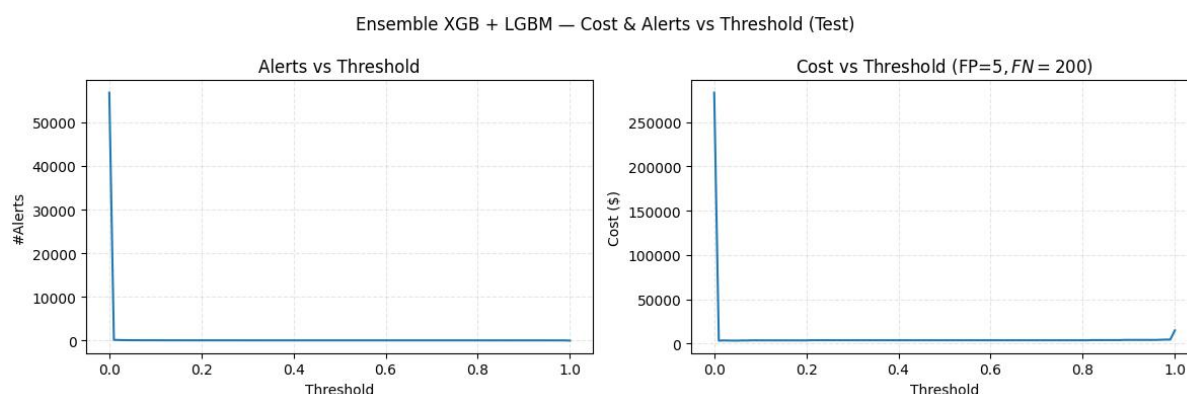
Đối với các mô hình ensemble, xu hướng tương tự cũng được quan sát.



Hình 18: Ensemble LGBM + RF - Cost & Alerts vs Threshold (Test)



Hình 19: Ensemble RF + XGB - Cost & Alerts vs Threshold (Test)



Hình 20: Ensemble XGB + LGBM - Cost & Alerts vs Threshold (Test)

trong đó, mô hình LightGBM + Random Forest cho thấy đường chi phí mượt và ổn định hơn, phản ánh khả năng tổng quát hóa tốt hơn so với các mô hình đơn. Ngược lại, các tổ hợp khác không cải thiện đáng kể, cho thấy việc kết hợp mô hình không phải lúc nào cũng mang lại hiệu quả nếu thiếu tính đa dạng.

Tóm lại, phân tích PR Curve và các biểu đồ liên quan cho thấy Random Forest và ensemble LightGBM + Random Forest là hai cấu hình đạt hiệu năng tốt nhất. Đồng thời, kết quả cũng nhấn mạnh rằng việc lựa chọn threshold phù hợp đóng vai trò quyết định trong việc tối ưu chi phí và hiệu quả vận hành của hệ thống phát hiện gian lận.

Chương 5. Triển khai hệ thống

5.1. Tổng quan hệ thống

Trong nghiên cứu này, tổng quan hệ thống web được xây dựng theo kiến trúc tách biệt giữa backend và frontend nhằm đảm bảo tính mở rộng và khả năng triển khai mô hình học máy trong thực tế. Hệ thống sử dụng FastAPI ở phía backend để triển khai các mô hình phát hiện gian lận giao dịch, đồng thời cung cấp các RESTful API phục vụ cho quá trình suy luận (inference). Phía frontend được phát triển bằng React, đóng vai trò xây dựng giao diện người dùng trực quan, cho phép nhập dữ liệu giao dịch và hiển thị kết quả dự đoán một cách trực quan và thân thiện.

Trong kiến trúc tổng thể, backend FastAPI chịu trách nhiệm xử lý dữ liệu đầu vào, thực hiện các bước tiền xử lý cần thiết và đưa dữ liệu vào mô hình ensemble (LGBM và XGBoost) đã được huấn luyện trước đó. Sau khi nhận được kết quả dự đoán từ mô hình, hệ thống sẽ trả phản hồi về phía client dưới dạng API response, bao gồm xác suất gian lận hoặc nhãn dự đoán tương ứng. Quá trình trao đổi dữ liệu giữa frontend và backend được thực hiện thông qua giao thức HTTP, đảm bảo tính linh hoạt và khả năng tích hợp cao.

Tổng quan hệ thống web này cho thấy khả năng tích hợp hiệu quả giữa học máy và phát triển ứng dụng web, trong đó mô hình không chỉ được huấn luyện và đánh giá trong môi trường offline mà còn được triển khai trong một hệ thống tương tác thực tế. Điều này giúp tăng tính ứng dụng của nghiên cứu, đồng thời cho phép kiểm chứng hiệu năng mô hình trong điều kiện sử dụng gần với thực tế.

5.2. Backend (API)

Về backend, tôi xây dựng hệ thống backend bằng FastAPI nhằm triển khai mô hình phát hiện gian lận giao dịch và cung cấp các API phục vụ cho việc xử lý dữ liệu và suy luận. Hệ thống được thiết kế theo kiến trúc phân tầng, bao gồm các thành phần chính: kết nối cơ sở dữ liệu, xử lý dữ liệu, suy luận mô hình và cập nhật kết quả.

Dữ liệu giao dịch được lưu trữ trong cơ sở dữ liệu MySQL và được truy xuất thông qua SQLAlchemy, giúp đảm bảo khả năng kết nối ổn định và linh hoạt giữa ứng

dụng và database. Các truy vấn được sử dụng để lấy dữ liệu phục vụ phân tích, đồng thời hỗ trợ phân trang và lọc theo trạng thái giao dịch. Sau khi được truy xuất, dữ liệu được xử lý và tổng hợp nhằm phục vụ cho cả mục đích dự đoán và phân tích thống kê.

Trong quá trình suy luận, dữ liệu đầu vào được tiền xử lý và tạo thêm các đặc trưng mở rộng nhằm cải thiện hiệu suất mô hình, bao gồm các đặc trưng về thời gian, hành vi giao dịch và biến đổi phi tuyến. Các đặc trưng này sau đó được đưa vào mô hình ensemble (Random Forest, XGBoost, LightGBM) để thực hiện dự đoán. Kết quả dự đoán bao gồm xác suất gian lận, nhãn phân loại và mức độ rủi ro, được cập nhật ngược trở lại cơ sở dữ liệu.

Bên cạnh chức năng dự đoán, backend còn cung cấp các API phục vụ thống kê và phân tích dữ liệu, chẳng hạn như tổng số giao dịch, tổng giá trị giao dịch theo trạng thái và phân bố giao dịch theo khung giờ. Các kết quả này được chuẩn hóa và trả về dưới dạng JSON để phục vụ hiển thị trên giao diện React.

Tổng thể, backend đóng vai trò trung tâm trong hệ thống, đảm nhiệm toàn bộ quá trình từ truy xuất dữ liệu, xử lý, suy luận mô hình đến cung cấp kết quả cho frontend, đảm bảo tính tự động hóa và khả năng triển khai thực tế của mô hình phát hiện gian lận.

5.3. Frontend

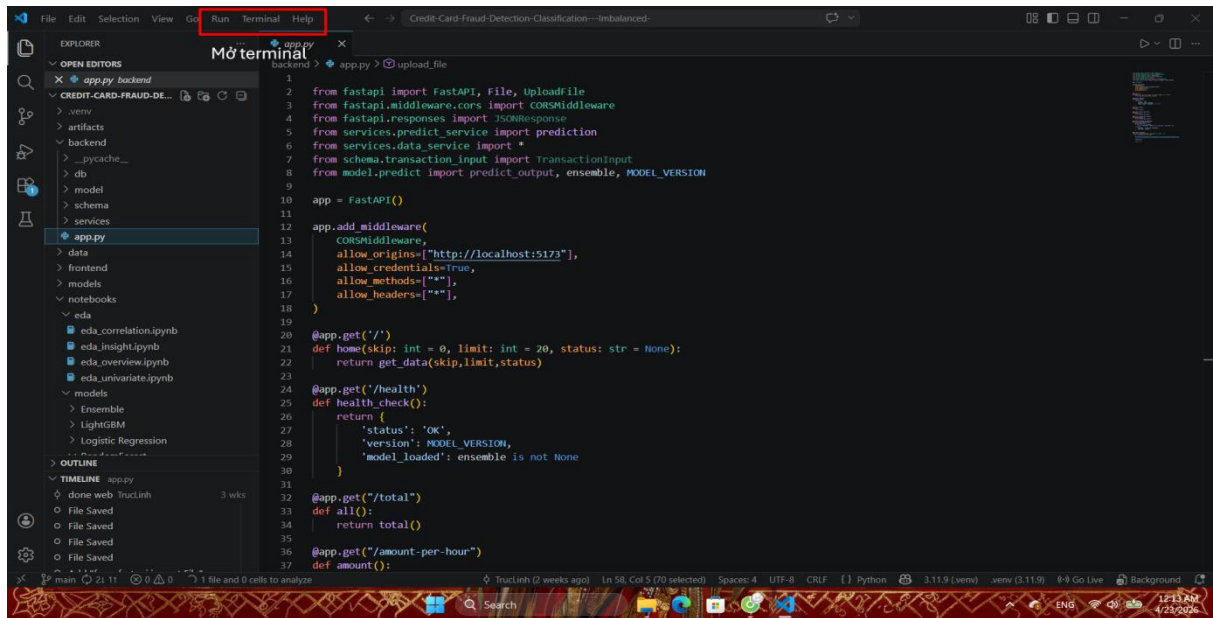
Tôi sử dụng ReactJS để xây dựng giao diện người dùng cho hệ thống phát hiện gian lận giao dịch. React được lựa chọn nhờ khả năng xây dựng giao diện dạng component, giúp tái sử dụng mã nguồn, dễ bảo trì và mở rộng. Bên cạnh đó, React hỗ trợ cập nhật giao diện theo cơ chế Virtual DOM, giúp tối ưu hiệu năng khi hiển thị dữ liệu động như bảng giao dịch hoặc biểu đồ thống kê.

Frontend đóng vai trò là lớp tương tác giữa người dùng và hệ thống backend. Người dùng có thể nhập dữ liệu giao dịch hoặc xem danh sách kết quả dự đoán thông qua giao diện React. Các thao tác này được gửi đến backend FastAPI thông qua các HTTP request (REST API), chủ yếu là phương thức GET và POST. Dữ liệu được truyền dưới định dạng JSON để đảm bảo tính tương thích và dễ xử lý giữa hai hệ thống.

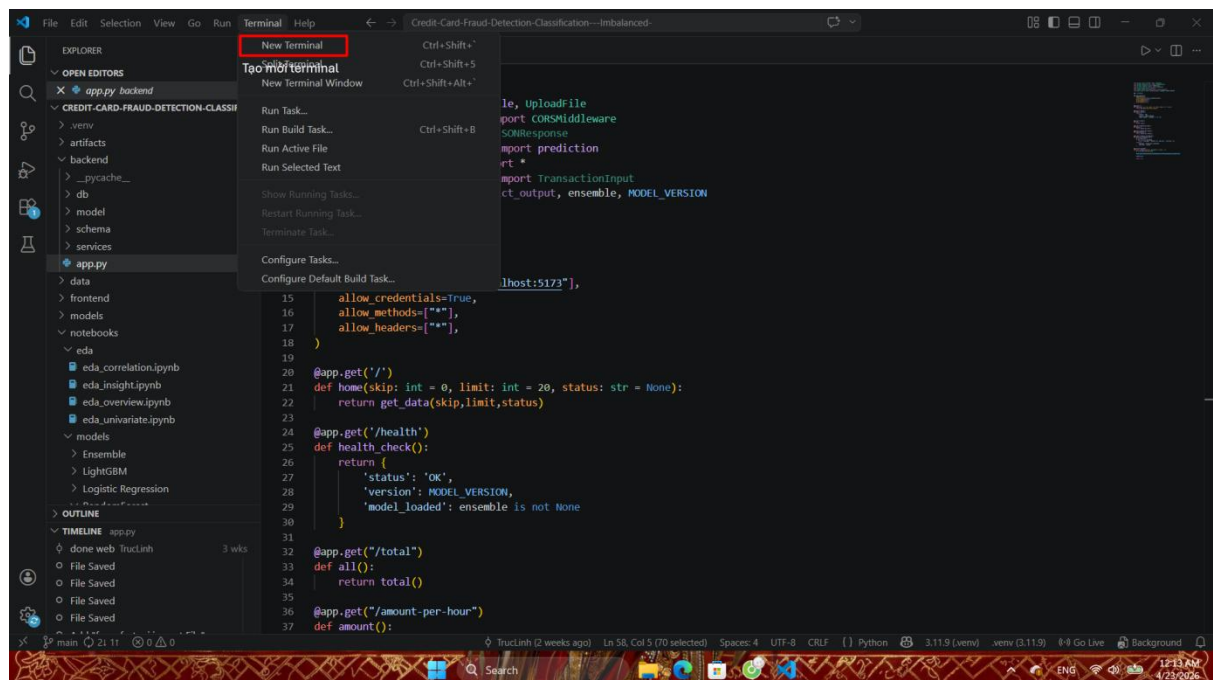
Sau khi nhận request từ frontend, FastAPI xử lý dữ liệu, thực hiện suy luận từ mô hình học máy và trả kết quả về lại frontend. React sau đó nhận phản hồi và cập nhật giao diện tương ứng theo thời gian thực, ví dụ như hiển thị xác suất gian lận, trạng thái giao dịch hoặc các thống kê liên quan. Cách kết nối này giúp tách biệt rõ

ràng giữa frontend và backend, đồng thời tạo ra một hệ thống linh hoạt, dễ mở rộng và dễ triển khai trong thực tế.

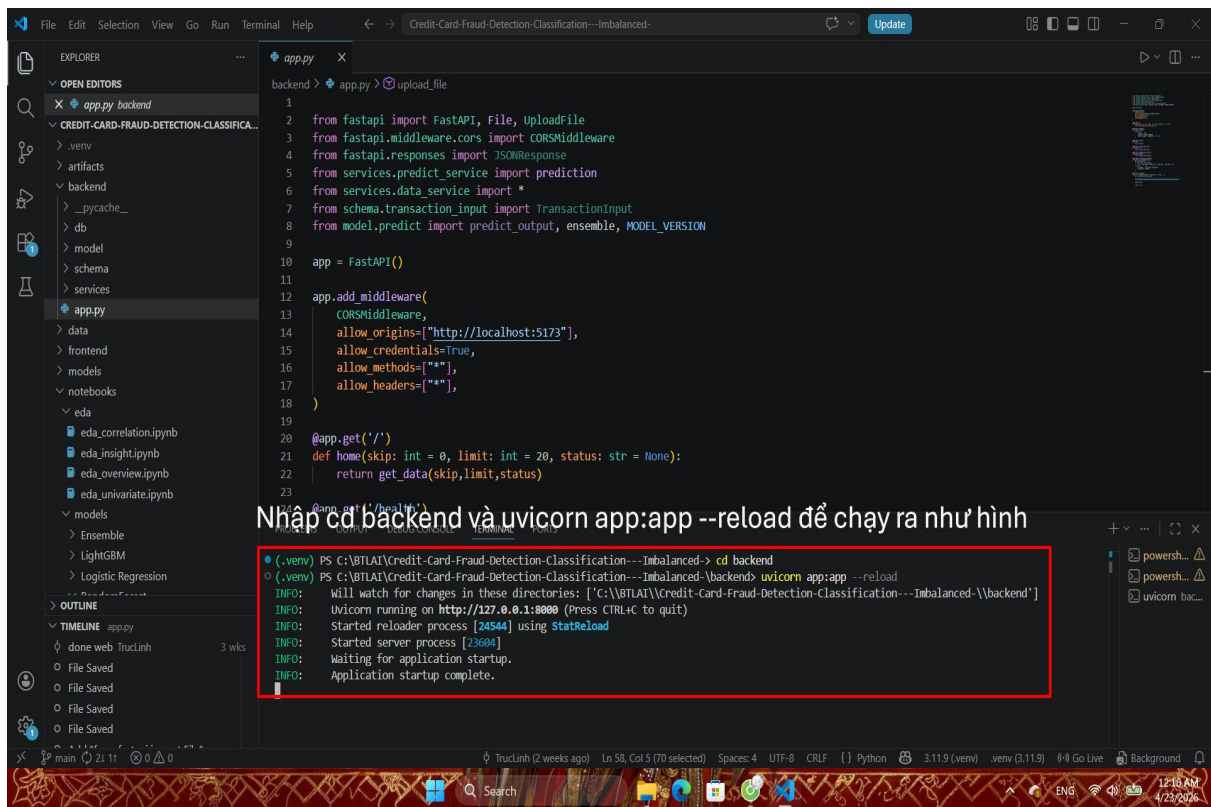
5.4. Demo hệ thống



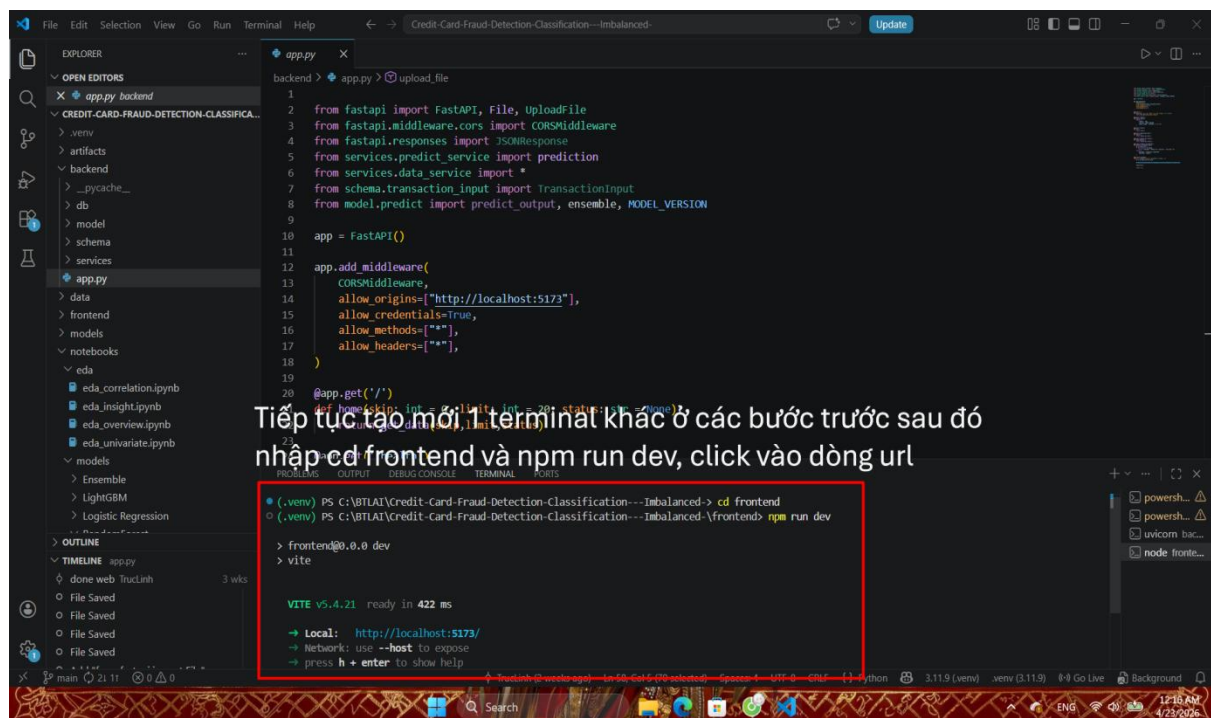
Hình 21: Ảnh Demo 1



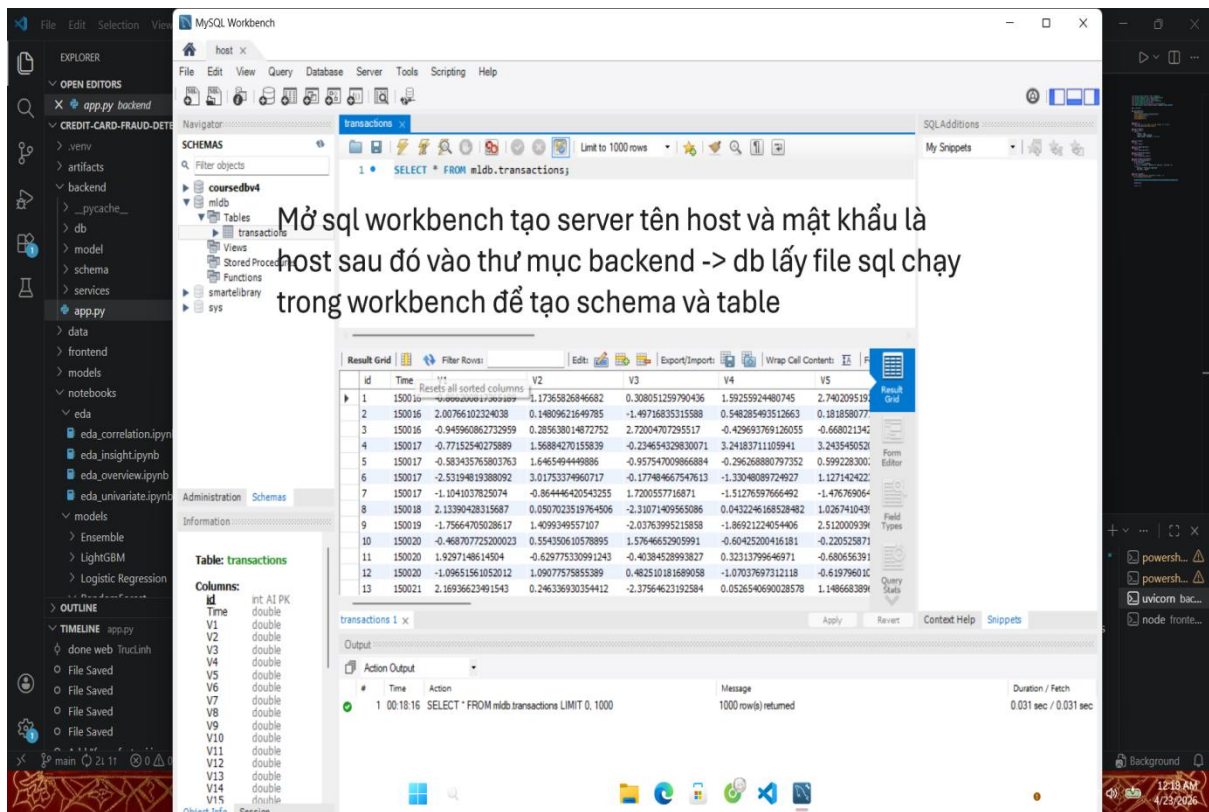
Hình 22: Ảnh Demo 2



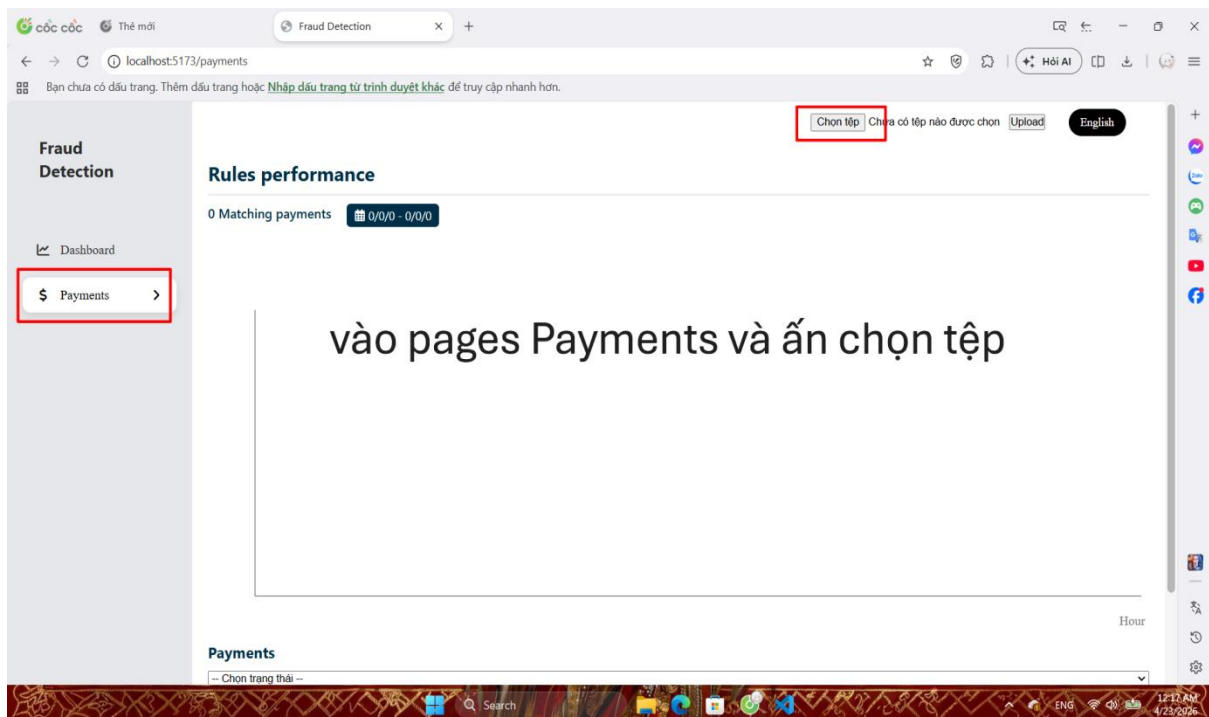
Hình 23: Ảnh Demo 3



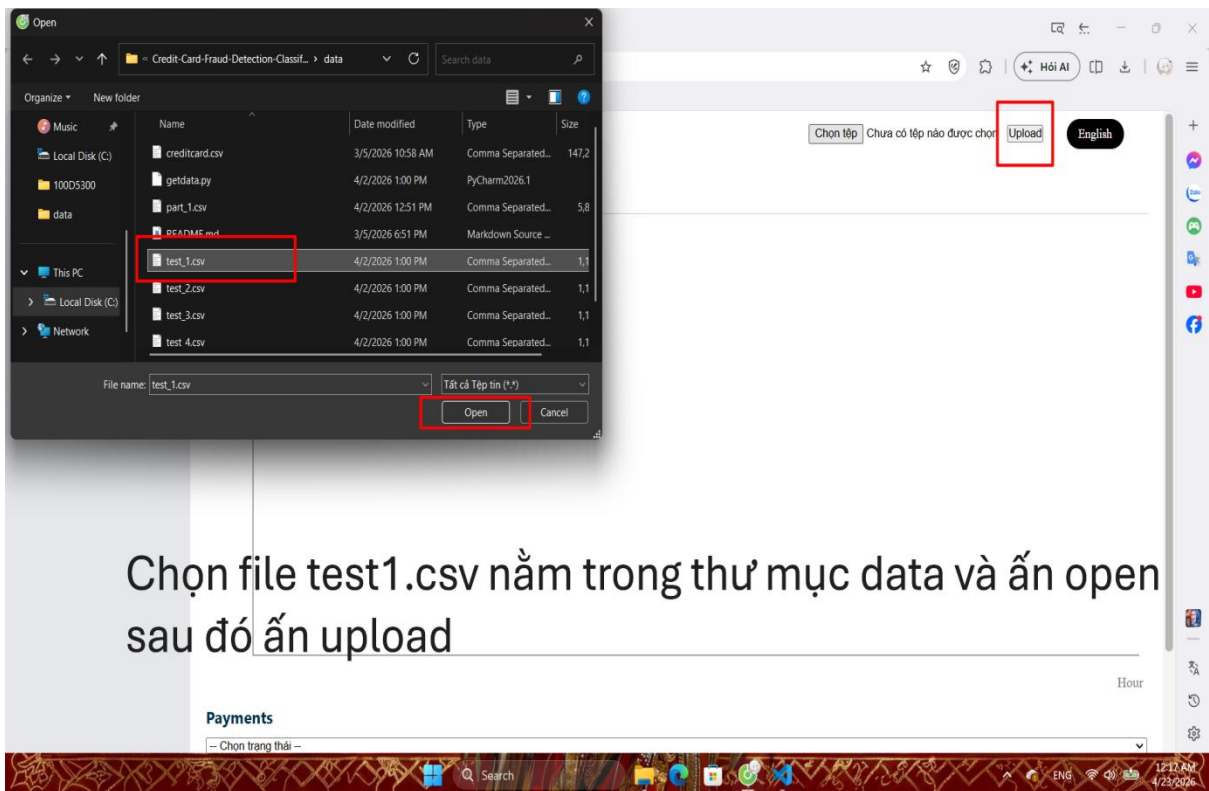
Hình 24: Ảnh Demo 4



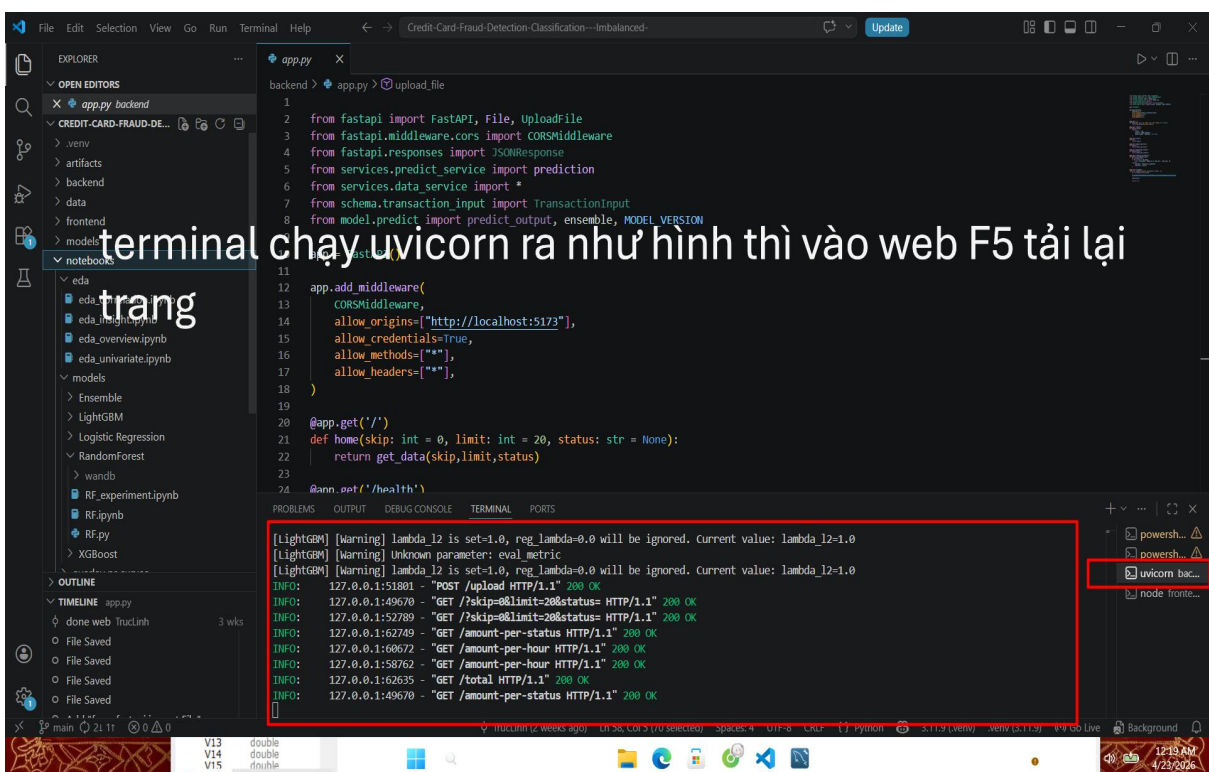
Hình 25: Ảnh Demo 5



Hình 26: Ảnh Demo 6



Hình 27: Ảnh Demo 7



Hình 28: Ảnh Demo 8